AI speed. Human accountability.

CREDSHIELDS · SMART CONTRACT AUDIT

Smart Contract Audit

NettyWorth

This report presents the findings of a smart contract security audit conducted against NettyWorth Smart Contracts V3. Findings were identified through manual review of the contract source by the CredShields' security team.

8	10	14	8	0	40
CRITICAL	HIGH	MEDIUM	LOW	INFO	TOTAL FINDINGS

CredShields Technologies Pte. Ltd. · Singapore
credshields.com · info@credshields.com

CONFIDENTIAL

/ CONTENTS

Table of Contents

01 Executive Summary

02 Vulnerability Summary

03 Detailed Findings

C001	Discount + Buyback-boost promo code stack yields guaranteed profit per pack	CRITICAL
C002	AcquireDefaultedAsset credits totalDeposited without USDC arriving phantom liquidity	CRITICAL
C003	Direct theft of user's NFT	CRITICAL
C004	Marketplace's loan settlement branch skips listing-seller authorization anyone can force-sell any borrower's collateral	CRITICAL
C005	Signed auction cannot be revoked cancelNonce does not gate commitBid; leaked auction signature stays valid post cancellation	CRITICAL
C006	PackMachine.fulfillRandomness nested IBuybackPool.registerToken revert kills the VRF callback; user pays for a pack that is never delivered	CRITICAL
C007	AssetNFT.batchBurn on Held-state collateral desyncs AssetLendingPool and P2PTradeEscrow accounting; loss is permanent	CRITICAL
C008	BuybackPool retroactive model switch (AmountSpent □ FMV + bearer fmv=1 quote) drains pool	CRITICAL
H001	Just-in-time lender deposit sandwich on loan repayment	HIGH
H002	FMV quote is a bearer instrument signer-binding never finished	HIGH
H003	Bundle loan over-release marketplace sale of one token releases the entire collateral bundle	HIGH
H004	buyWithSignature SignedListing is a bearer instrument front-running steals private deals and griefs intended buyers	HIGH
H005	Cross-Collection tokenId Collision latent collateral heist activates on multi-collection allowlist	HIGH
H006	BuybackPool._executeBuyback FMV-quote nonce not reset on registerToken; leaked stale quote drains pool after model flip	HIGH
H007	AssetNFT.initiateShipment no maxFee slippage; admin setAppraisal front-run drains user's USDC approval	HIGH
H008	AssetLendingPool._collectOriginationFee live-read at disbursement; admin setOriginationFee front-run shrinks borrower's disbursement to zero	HIGH
H009	AssetLendingPool.financeMarketplacePurchase skips listing.buyer enforcement private listing hijack	HIGH
H010	Seller nonce cancellation asymmetry marketplace.cancelNonce burns only usedNonces, leaving pool.financeNonces redeemable	HIGH

M001	Stranded pack open VRF callback can fail and consume payment without delivering cards or refund	MEDIUM
M002	borrowerLoans array is append-only unbounded growth, no closing-path cleanup	MEDIUM
M003	Mutable lenderShareBps retroactively reprices in-flight loan interest	MEDIUM
M004	PackMachine config setters not pause-gated race window between user payment and VRF callback	MEDIUM
M005	Marketplace allowlist setter has no compatibility canary fee-on-transfer token addition will brick settlement	MEDIUM
M006	Discount promo code is not scoped to a PackMachine cross-machine misuse + griefing	MEDIUM
M007	Auction can be cancelled after a bidder commits griefing the bidder's gas + denying the NFT	MEDIUM
M008	Default forfeits all loan interest lenders and protocol get only principal back	MEDIUM
M009	financeMarketplacePurchase allows self-financing of pool's defaulted collateral DoS of default resolution + permanent accounting corruption	MEDIUM
M010	PackMachineFactory.beforeTransfer / .afterTransfer low-level .call() silently swallows validator reverts; selector 0x931820d7 does not match canonical CTS	MEDIUM
M011	PackMachine.resetEffectivePrizePoolSize ignores in-flight VRF reservations; post-reset opens oversell inventory, later callbacks emit CardFailed(tokenId=0)	MEDIUM
M012	NettyWorthMarketplace.hashAuction returns domain-separated hash but commitBid uses bare struct hash; documented API always fails	MEDIUM
M013	Transfer-validator revert in PackMachine.fulfillRandomness permanently locks user USDC and stucks pool accounting	MEDIUM
M014	BuybackPool pause silently strips buyback rights from every card won during the pause window	MEDIUM
L001	missing zero-address check + non-pause-gated; pool unset silently disables buyback registration	LOW
L002	PackMachine.rescueERC20 can drain any ERC-20 balance admin foot-gun	LOW
L003	BuybackPool _redeposit silently no-ops if source machine deregistered NFT stuck, payout already made	LOW
L004	oncePerUser discount code burnable by anyone with leaked OpenPack signature	LOW
L005	NettyWorthMarketplace._executeSale royalty deducted from seller proceeds but not paid when royaltyReceiver \== address(0); USDC stranded	LOW
L006	AssetLendingPool.setDefaultLifecycleConfig retroactive window change contradicts docstring; admin can flip phase for in-flight defaults	LOW
L007	BuybackPool.initializeV2 unprotected reinitializer(2); V2 slot front-runnable	LOW
L008	BuybackPool._executeBuyback silently switched BuybackExecuted.payout and totalPaidOut from net to gross; off-chain integrators overstate user proceeds	LOW

– Lead Reviewers & Signatories

/ 01: THE PICTURE AT A GLANCE

Executive Summary

CredShields conducted a security audit of the NettyWorth Smart Contracts V3, identifying a total of **40 findings** across various severity levels. The initial assessment revealed **8 Critical, 10 High, 14 Medium, and 8 Low** severity issues, with no informational findings.

The Critical findings highlight severe vulnerabilities that could lead to significant financial loss or complete system compromise. These include a business logic flaw allowing **guaranteed profit per pack** through promo code stacking, **direct theft of user NFTs**, and a critical access control issue where **anyone could force-sell a borrower's collateral** on the marketplace. Other critical issues noted include a bug where users could **pay for packs that are never delivered** due to VRF callback failure, and a retroactive model switch in the BuybackPool that could **drain the pool**.

High severity findings also present substantial risks, such as **just-in-time lender deposit sandwich attacks** on loan repayment, **signature replay attacks** due to incomplete signer-binding for FMV quotes, and front-running vulnerabilities that allow the **theft of private deals** or draining of user USDC approvals. These issues required immediate attention to protect user funds and system integrity.

Following the audit, NettyWorth diligently addressed all identified vulnerabilities. A retest confirmed that **all 40 findings have been fixed**, demonstrating a strong commitment to the security of their smart contract ecosystem.

Audited scope

https://github.com/nettyworth/NettyWorthV3/tree/91c127cf68074f3a0abe3167bb2e7b7b4dc95503

Retested scope

https://github.com/nettyworth/NettyWorthV3/tree/3103a36ea22bacdd2c21491eead56244a1b2c969

Assessment overview

CredShields performed a comprehensive smart contract security audit for NettyWorth's Smart Contracts V3. This engagement involved a detailed manual code review, architectural analysis, and the application of automated tools to identify potential security vulnerabilities, business logic flaws, and deviations from intended behavior. Our analysis focused on critical areas such as access control mechanisms, financial accounting integrity, potential for economic exploits, reentrancy vectors, denial-of-service attack surfaces, and adherence to best practices in smart contract development.

The audit identified a total of 40 findings across various severity levels, all of which NettyWorth successfully remediated. The most severe issues, categorized as Critical (8 findings), pointed to vulnerabilities that could lead to significant financial loss, direct asset theft, or severe protocol instability. For instance, **Discount + Buyback-boost promo code stack yields guaranteed profit per pack** highlighted an exploit where specific promo code combinations could allow users to generate guaranteed profit. Similarly, **Direct theft of user's NFT** exposed a flaw in access control, enabling unauthorized asset transfer. Critical accounting inconsistencies were also found, such as **AcquireDefaultedAsset credits totalDeposited without USDC arriving**, which could create phantom liquidity, and **AssetNFT.batchBurn on Held-state collateral desyncs AssetLendingPool and P2PTradeEscrow accounting**, leading to permanent asset loss. Further critical issues included **Marketplace's loan settlement branch skips listing-seller authorization**, allowing forced sales of collateral, and **PackMachine.fulfillRandomness nested IBuybackPool.registerToken revert kills the VRF callback**, leading to user payment without service delivery.

Beyond the critical flaws, 10 High severity findings were discovered, many of which involved front-running opportunities and signature replay attacks. Examples include **Just-in-time lender deposit sandwich on loan repayment** and **buyWithSignature SignedListing is a bearer instrument front-running steals private deals**, both illustrating how malicious actors could exploit transaction ordering. Signature integrity was also a concern with **FMV quote is a bearer instrument signer-binding never finished**, which could enable replay attacks with stale quotes. Access control remained a theme, as seen in **AssetLendingPool.financeMarketplacePurchase skips listing.buyer enforcement**, allowing private listing hijack.

The audit also uncovered 14 Medium severity findings, often related to less immediate but still impactful business logic errors, potential denial-of-service vectors, or unoptimized contract patterns. Notable issues included **Stranded pack open VRF callback can fail and consume payment without delivering cards or refund**, indicating a risk of user funds being locked, and **PackMachine config setters not pause-gated**, creating race conditions. Other medium findings highlighted unbounded array growth in `borrowerLoans` and potential DoS scenarios like **financeMarketplacePurchase allows self-financing of pool's defaulted collateral**, which could corrupt accounting. Finally, 8 Low severity findings were identified, encompassing minor security concerns, missing input validations, potential admin foot-guns like **PackMachine.rescueERC20 can drain any ERC-20 balance**, and issues affecting off-chain integrations or user experience. The successful retest of all 40 findings confirms their resolution, significantly enhancing the security posture of the NettyWorth Smart Contracts V3.

ENGAGEMENT DETAILS

CLIENT	NettyWorth
ASSESSMENT TYPE	Smart contract security audit
AUDIT	NettyWorth Smart Contracts V3
REPORT DATE	14 Jul 2026
START DATE	15 Jun 2026
END DATE	02 Jul 2026
RETEST DATE	14 Jul 2026
TOTAL FINDINGS	40
REVIEWER	Shashank (shashank@credshields.com) · CredShields

/ FINDING SEVERITY DISTRIBUTION

8	10	14	8	0
CRITICAL	HIGH	MEDIUM	LOW	INFO



/ 02: AT A GLANCE

Vulnerability Summary

Every finding from this assessment, ordered by severity. Follow an ID for the full write-up.

ID	VULNERABILITY	SEVERITY	CATEGORY	STATUS
C001	Discount + Buyback-boost promo code stack yields guaranteed profit per pack	CRITICAL	Business Logic (SC02-LogicErrors)	✓ FIXED
C002	AcquireDefaultedAsset credits totalDeposited without USDC arriving phantom liquidity	CRITICAL	Inconsistent Accounting (SCWE-010)	✓ FIXED
C003	Direct theft of user's NFT	CRITICAL	Access Control (SCWE-016)	✓ FIXED
C004	Marketplace's loan settlement branch skips listing-seller authorization anyone can force-sell any borrower's collateral	CRITICAL	Uncaught Exceptions (SCWE-004)	✓ FIXED
C005	Signed auction cannot be revoked cancelNonce does not gate commitBid; leaked auction signature stays valid post cancellation	CRITICAL	Access Control (SCWE-016)	✓ FIXED
C006	PackMachine.fulfillRandomness nested IBuybackPool.registerToken revert kills the VRF callback; user pays for a pack that is never delivered	CRITICAL	Uncaught Exceptions (SCWE-004)	✓ FIXED
C007	AssetNFT.batchBurn on Held-state collateral desyncs AssetLendingPool and P2PTradeEscrow accounting; loss is permanent	CRITICAL	Inconsistent Accounting (SCWE-010)	✓ FIXED
C008	BuybackPool retroactive model switch (AmountSpent □ FMV + bearer fmv=1 quote) drains pool	CRITICAL	Business Logic (SC02-LogicErrors)	✓ FIXED
H001	Just-in-time lender deposit sandwich on loan repayment	HIGH	Front-Running (SCWE-037)	✓ FIXED
H002	FMV quote is a bearer instrument signer-binding never finished	HIGH	Signature Replay Attacks (SCWE-055)	✓ FIXED
H003	Bundle loan over-release marketplace sale of one token releases the entire collateral bundle	HIGH	Uncaught Exceptions (SCWE-004)	✓ FIXED
H004	buyWithSignature SignedListing is a bearer instrument front-running steals private deals and griefs intended buyers	HIGH	Front-Running (SCWE-037)	✓ FIXED
H005	Cross-Collection tokenId Collision latent collateral heist activates on multi-collection allowlist	HIGH	Uncaught Exceptions (SCWE-004)	✓ FIXED
H006	BuybackPool._executeBuyback FMV-quote nonce not reset on registerToken; leaked stale quote drains pool after model flip	HIGH	Business Logic (SC02-LogicErrors)	✓ FIXED

ID	VULNERABILITY	SEVERITY	CATEGORY	STATUS
H007	AssetNFT.initiateShipment no maxFee slippage; admin setAppraisal front-run drains user's USDC approval	HIGH	Business Logic (SC02-LogicErrors)	✓ FIXED
H008	AssetLendingPool._collectOriginationFee live-read at disbursement; admin setOriginationFee front-run shrinks borrower's disbursement to zero	HIGH	Business Logic (SC02-LogicErrors)	✓ FIXED
H009	AssetLendingPool.financeMarketplacePurchase skips listing.buyer enforcement private listing hijack	HIGH	Access Control (SCWE-016)	✓ FIXED
H010	Seller nonce cancellation asymmetry marketplace.cancelNonce burns only usedNonces, leaving pool.financeNonces redeemable	HIGH	Access Control (SCWE-016)	✓ FIXED
M001	Stranded pack open VRF callback can fail and consume payment without delivering cards or refund	MEDIUM	Uncaught Exceptions (SCWE-004)	✓ FIXED
M002	borrowerLoans array is append-only unbounded growth, no closing-path cleanup	MEDIUM	Inconsistent Accounting (SCWE-010)	✓ FIXED
M003	Mutable lenderShareBps retroactively reprices in-flight loan interest	MEDIUM	Uncaught Exceptions (SCWE-004)	✓ FIXED
M004	PackMachine config setters not pause-gated race window between user payment and VRF callback	MEDIUM	Missing functionality (SCWE-006)	✓ FIXED
M005	Marketplace allowlist setter has no compatibility canary fee-on-transfer token addition will brick settlement	MEDIUM	Missing functionality (SCWE-006)	✓ FIXED
M006	Discount promo code is not scoped to a PackMachine cross-machine misuse + griefing	MEDIUM	Access Control (SCWE-016)	✓ FIXED
M007	Auction can be cancelled after a bidder commits griefing the bidder's gas + denying the NFT	MEDIUM	Business Logic (SC02-LogicErrors)	✓ FIXED
M008	Default forfeits all loan interest lenders and protocol get only principal back	MEDIUM	Business Logic (SC02-LogicErrors)	✓ FIXED
M009	financeMarketplacePurchase allows self-financing of pool's defaulted collateral DoS of default resolution + permanent accounting corruption	MEDIUM	Denial of Service (SCWE-087)	✓ FIXED
M010	PackMachineFactory.beforeTransfer / .afterTransfer low-level .call() silently swallows validator reverts; selector 0x931820d7 does not match canonical CTS	MEDIUM	Uncaught Exceptions (SCWE-004)	✓ FIXED
M011	PackMachine.resetEffectivePrizePoolSize ignores in-flight VRF reservations; post-reset opens oversell inventory, later callbacks emit CardFailed(tokenId=0)	MEDIUM	Inconsistent Accounting (SCWE-010)	✓ FIXED
M012	NettyWorthMarketplace.hashAuction returns domain-separated hash but commitBid uses bare struct hash; documented API always fails	MEDIUM	Business Logic (SC02-LogicErrors)	✓ FIXED

ID	VULNERABILITY	SEVERITY	CATEGORY	STATUS
M013	Transfer-validator revert in PackMachine.fulfillRandomness permanently locks user USDC and stucks pool accounting	MEDIUM	Denial of Service (SCWE-087)	✓ FIXED
M014	BuybackPool pause silently strips buyback rights from every card won during the pause window	MEDIUM	Business Logic (SC02-LogicErrors)	✓ FIXED
L001	missing zero-address check + non-pause-gated; pool unset silently disables buyback registration	LOW	Lack of Input Validation (SC05-Lack Of Input Validation)	✓ FIXED
L002	PackMachine.rescueERC20 can drain any ERC-20 balance admin foot-gun	LOW	Centralization Risk	✓ FIXED
L003	BuybackPool _redeposit silently no-ops if source machine deregistered NFT stuck, payout already made	LOW	Missing Event Emission (SCWE-063)	✓ FIXED
L004	oncePerUser discount code burnable by anyone with leaked OpenPack signature	LOW	Signature Replay Attacks (SCWE-055)	✓ FIXED
L005	NettyWorthMarketplace._executeSale royalty deducted from seller proceeds but not paid when royaltyReceiver \== address(0); USDC stranded	LOW	—	✓ FIXED
L006	AssetLendingPool.setDefaultLifecycleConfig retroactive window change contradicts docstring; admin can flip phase for in-flight defaults	LOW	Business Logic (SC02-LogicErrors)	✓ FIXED
L007	BuybackPool.initializeV2 unprotected reinitializer(2); V2 slot front-runnable	LOW	Access Control (SCWE-016)	✓ FIXED
L008	BuybackPool._executeBuyback silently switched BuybackExecuted.payout and totalPaidOut from net to gross; off-chain integrators overstate user proceeds	LOW	Uncaught Exceptions (SCWE-004)	✓ FIXED

/ 03: DETAILED FINDINGS

40 findings across the assessment.

Findings are ordered by severity.

C001 · BUSINESS LOGIC (SC02-LOGICERRORS)

CRITICAL

FIXED

Discount + Buyback-boost promo code stack yields guaranteed profit per pack

DESCRIPTION

PackMachine.fulfillRandomness registers each won card in BuybackPool with a cost basis of pricePerPack / cardsPerPack the undiscounted per-card price regardless of any discount the buyer actually paid in _handlePayment. When the buyer later sells the card back through BuybackPool._executeBuyback and applies a buyback boost code, the payout is pricePerCard × boostedBps / BPS calculated against the undiscounted basis. The buy side reduces the user's outlay; the sell side increases the user's payout; the two never reconcile. With P=100, N=5, discount=2500, boost=9800, the buyer pays 75, opens 5 cards each registered at basis 20, and sells them back for 5 × 19.6 \= 98 a guaranteed 23 USDC profit per pack. Every cell of the ({1000,1500,2000,2500} × {9000,9500,9800}) matrix other than (1000, 9000) is buyer profitable, so the attack does not require unusual code parameters.

AFFECTED CODE

- https://github.com/nettyworth/NettyWorthV3/blob/e2490ee60c62454780fe9e86fed9d2ecfe35a954/contracts/PackMachine.sol#L444-L449
- https://github.com/nettyworth/NettyWorthV3/blob/e2490ee60c62454780fe9e86fed9d2ecfe35a954/contracts/PackMachine.sol#L848-L894
- https://github.com/nettyworth/NettyWorthV3/blob/e2490ee60c62454780fe9e86fed9d2ecfe35a954/contracts/BuybackPool.sol#L580-L666

BUSINESS IMPACT

Direct drain of BuybackPool. At scale the pool drains to zero, after which honest sellers' buyback calls revert with BuybackPool__InsufficientBalance and the buyback feature is bricked for everyone.

RECOMMENDED NEXT STEP

Snapshot the paid per-card price in pendingOpens as discountedPrice / cards. and register that snapshot as a basis in fulfillRandomness.

RETEST

Fixed
This issue is fixed by snapshotting escrowedAmount (post-discount USDC) in PendingOpen and computing basisPerCard \= pending.escrowedAmount / pending.cardsCount in fulfillRandomness, then passing that discounted per-card basis to IBuybackPool.registerToken the discount+boost arbitrage is eliminated.

C002 · INCONSISTENT ACCOUNTING (SCWE-010)

CRITICAL

AcquireDefaultedAsset credits totalDeposited without USDC arriving phantom liquidity

✓ FIXED

DESCRIPTION

_initiateDefault debits totalDeposited -= principal to absorb the immediate loss. acquireDefaultedAsset (Phase-1 recovery) then recredits totalDeposited += outstandingValue but recovery happens via NFT-recycling into a PackMachine, not a USDC inflow. The pool's accounting now claims liquidity it doesn't hold. The available liquidity check (amount ≤ totalDeposited − totalBorrowed) treats this phantom amount as borrowable and withdrawable. Only Phase-2/3 (purchaseDefaultedAsset) actually pulls USDC. After several acquireDefaultedAsset resolutions, the gap between the pool's claimed totalDeposited and its real USDC balance grows monotonically, until the last lender to withdraw hits a safeTransfer revert from insufficient real balance even though their on-chain lenderDeposits[user] shows full balance.

AFFECTED CODE

- https://github.com/nettyworth/NettyWorthV3/blob/e2490ee60c62454780fe9e86fed9d2ecfe35a954/contracts/AssetLendingPool.sol#L487-L533
- https://github.com/nettyworth/NettyWorthV3/blob/e2490ee60c62454780fe9e86fed9d2ecfe35a954/contracts/AssetLendingPool.sol#L871-L910
- https://github.com/nettyworth/NettyWorthV3/blob/e2490ee60c62454780fe9e86fed9d2ecfe35a954/contracts/AssetLendingPool.sol#L543-L574

BUSINESS IMPACT

The lender silently becomes unredeemable. Breaks the protocol's "lenders can always exit" invariant via deterministic accounting drift.

RECOMMENDED NEXT STEP

It is suggested to not credit phantom liquidity. Track NFT-backed recovery in a separate counter that does NOT contribute to getAvailableLiquidity. Credit totalDeposited only when real USDC arrives (via purchaseDefaultedAsset or via a BuybackPool callback that returns USDC after a pack sell-through).

RETEST

Fixed
This issue is fixed by pulling principal + interest in real USDC from the finance wallet atomically inside acquireDefaultedAsset via _resolveAndRecredit before the totalDeposited credit becomes spendable the credit is now backed by real cash instead of phantom NFT-backed value.

C003 · ACCESS CONTROL (SCWE-016)

Direct theft of user's NFT

CRITICAL

✓ FIXED

DESCRIPTION

financeMarketplacePurchase accepts seller as a user controlled parameter and pulls the NFT via assetNFT.transferFrom(seller, address(this), tokenId) using only the seller's ERC-721 approval to the pool as authorization. ERC-721 approval is a transfer permission, not a sale consent at any specific price to any specific buyer. Any pre borrow approval (typical 12-second mempool window between approve and borrow on Ethereum) or any active setApprovalForAll(pool, true) (common DeFi UX pattern) can be hijacked: an attacker calls financeMarketplacePurchase(tokenId, depositAmount=minimum, termId, seller=Alice) and atomically becomes the borrower, with the NFT now collateralizing the attacker's loan. The seller receives appraisalValue in cash (which they may not have wanted to sell at), and the attacker can immediately relist the NFT at market value through the marketplace, capturing the appraisal-vs-market gap. The 7-day default maxAppraisalAge window makes this profitable on any NFT in a rising market.

AFFECTED CODE

- https://github.com/nettyworth/NettyWorthV3/blob/e2490ee60c62454780fe9e86fed9d2ecfe35a954/contracts/AssetLendingPool.sol#L272-L337
- https://github.com/nettyworth/NettyWorthV3/blob/e2490ee60c62454780fe9e86fed9d2ecfe35a954/contracts/AssetLendingPool.sol#L306
- https://github.com/nettyworth/NettyWorthV3/blob/e2490ee60c62454780fe9e86fed9d2ecfe35a954/contracts/AssetLendingPoolConfig.sol#L522-L538

BUSINESS IMPACT

Forced sale of any approving user's NFT at appraisal value. With stale appraisal + active market, attacker profit ≈ realValue − appraisalValue − fees − interest per attack; victim loses the appraisal-vs-market spread permanently.

RECOMMENDED NEXT STEP

It is recommended an EIP-712 FinancePurchase(uint256 tokenId, uint256 minDeposit, address buyer, uint256 nonce, uint256 deadline) signature from the seller, with per-seller nonce replay protection. Alternatively restrict financeMarketplacePurchase to msg.sender \== marketplace so the marketplace's existing SignedListing becomes the consent layer.

RETEST

Fixed
By requiring an EIP-712 seller signature on the SignedListing and verifying it via ECDSA.recover(digest, sig) \== listing.seller inside financeMarketplacePurchase the function no longer treats the ERC-721 approval as sale consent.

C004 · UNCAUGHT EXCEPTIONS (SCWE-004)

CRITICAL

FIXED

Marketplace's loan settlement branch skips listing-seller authorization anyone can force-sell any borrower's collateral

DESCRIPTION

The _executeSale branches at L531: if loanDebt > 0, it calls settleLoanRepaymentOnSale (which makes the POOL deliver the NFT directly to the buyer); else it does IAssetNFT(collection).transferFrom(seller, buyer, tokenId) (which ERC-721 enforces seller ownership). For non-loan sales (branch 7b), ERC-721's transferFrom reverts unless the seller \== owner, providing implicit authorization. For collateralized NFTs (branch 7a), this implicit check disappears the pool transfers from its own custody and never checks who claimed to be the seller. The seller field comes from listing.seller (or state.seller for auctions), is only verified to have signed the EIP-712 struct, and is finally used at L542-544 to receive sellerProceeds. Anyone with any private key can sign a SignedListing for a victim's collateralized AssetNFT, pay loanDebt + fees through a sock-puppet buyer, pocket the surplus as sellerProceeds, and walk away with the victim's NFT in the buyer wallet while the victim (the actual loan borrower) gets the loan marked paid but receives no cash from the sale and loses their NFT. Each contract's books balance; no event flags the seller-vs-borrower mismatch; the victim only discovers the heist when their NFT is gone.

AFFECTED CODE

- https://github.com/nettyworth/NettyWorthV3/blob/e2490ee60c62454780fe9e86fed9d2ecfe35a954/contracts/NettyWorthMarketplace.sol#L474-L558
- https://github.com/nettyworth/NettyWorthV3/blob/e2490ee60c62454780fe9e86fed9d2ecfe35a954/contracts/NettyWorthMarketplace.sol#L531-L539
- https://github.com/nettyworth/NettyWorthV3/blob/e2490ee60c62454780fe9e86fed9d2ecfe35a954/contracts/NettyWorthMarketplace.sol#L542-L544
- https://github.com/nettyworth/NettyWorthV3/blob/e2490ee60c62454780fe9e86fed9d2ecfe35a954/contracts/NettyWorthMarketplace.sol#L169-L215
- https://github.com/nettyworth/NettyWorthV3/blob/e2490ee60c62454780fe9e86fed9d2ecfe35a954/contracts/NettyWorthMarketplace.sol#L321-L347
- https://github.com/nettyworth/NettyWorthV3/blob/e2490ee60c62454780fe9e86fed9d2ecfe35a954/contracts/AssetLendingPool.sol#L214-L258

BUSINESS IMPACT

Direct, single-transaction, permanent theft of any active loan's collateral by any unprivileged attacker.

RECOMMENDED NEXT STEP

Enforce listing.seller \== loan.borrower when the loan branch fires. In _executeSale, after fetching the loan.

RETEST

Fixed
This vulnerability has been fixed by enforcing seller \== loan.borrower in the active-loan branch of _settleSale and reverting with Marketplace__SellerNotBorrower before calling settleLoanRepaymentOnSale an unauthorized seller can no longer force-close a borrower's collateralized listing.

C005 · ACCESS CONTROL (SCWE-016)

CRITICAL

FIXED

Signed auction cannot be revoked cancelNonce does not gate commitBid; leaked auction signature stays valid post cancellation

DESCRIPTION

NettyWorthMarketplace exposes two off-chain signed order flows that share the same usedNonces map: fixed-price buyWithSignature and English-auction commitBid. buyWithSignature correctly enforces the seller's nonce; it reverts if usedNonces[listing.seller][listing.nonce] is set and marks it after use, so cancelNonce(N) from the seller invalidates any leaked SignedListing. commitBid handles only the bidder's nonce (checks and burns usedNonces[bid.bidder][bid.nonce]) and never consults usedNonces[auction.seller][auction.nonce]. The seller therefore has no on-chain path to invalidate a leaked SignedAuction before it is materialised cancelAuction only works after state.exists \== true, i.e. after at least one bid has already committed to the auction. Concrete trace: Alice signs SignedAuction{seller=Alice, tokenId=42, reservePrice=50, endTime=T+7d, nonce=N} for what should have been reservePrice=500 (typo); signature leaks. Alice calls cancelNonce(N) succeeds but has no effect on the auction. Bob obtains the leaked bytes, calls commitBid(auction, sig, bid at 50); auction materialises. After endTime, Bob calls settleAuction(auctionId) □ NFT transferred Alice □ Bob at 50 USDC. Alice loses the full 450 USDC market spread.

AFFECTED CODE

- https://github.com/nettyworth/NettyWorthV3/blob/e2490ee60c62454780fe9e86fed9d2ecfe35a954/contracts/NettyWorthMarketplace.sol#L192-L215
- https://github.com/nettyworth/NettyWorthV3/blob/e2490ee60c62454780fe9e86fed9d2ecfe35a954/contracts/NettyWorthMarketplace.sol#L222-L316
- https://github.com/nettyworth/NettyWorthV3/blob/e2490ee60c62454780fe9e86fed9d2ecfe35a954/contracts/NettyWorthMarketplace.sol#L356-L374

BUSINESS IMPACT

Any leaked or improperly-shared SignedAuction remains fully redeemable at the reserve price regardless of the seller's revocation attempt. Attacker with a leaked signature (or a UI aggregator that publishes and then the seller retracts) can materialise the auction and settle at the reserve, capturing the seller-vs-market spread with no seller consent. Direct NFT theft primitive on any signed-but-unposted auction.

RECOMMENDED NEXT STEP

In commitBid, on the first-bid materialisation branch (if (!state.exists)), consume the seller's nonce mirroring the fixed-price flow: revert if $.usedNonces[auction.seller][auction.nonce] is set, and mark it true when materialising the auction. This makes cancelNonce(N) symmetric across both signed-order paths.

RETEST

Fixed
This issue has been fixed by consuming the seller's nonce inside the first-bid materialisation branch of commitBid the code now reverts with Marketplace__NonceUsed(auction.seller, auction.nonce) when the leaked SignedAuction's nonce is already burned and marks it used on materialisation, so cancelNonce(N) from the seller now invalidates both fixed-price listings and auctions symmetrically.

C006 · UNCAUGHT EXCEPTIONS (SCWE-004)

CRITICAL

PackMachine.fulfillRandomness nested IBuybackPool.registerToken revert kills the VRF callback; user pays for a pack that is never delivered

✓ FIXED

DESCRIPTION

fulfillRandomness wraps the ERC-721 transfer of each drawn card in a try { IERC721.transferFrom(…) { … } } catch { … } block. Inside the success arm critically NOT inside the tried expression it calls IBuybackPool(pool).registerToken(tokenId, pricePerCard, tier, address(this)). Solidity's try/catch scope covers only the tried external-call expression, so any revert thrown by registerToken propagates uncaught and unwinds the entire fulfillRandomness execution. Because BuybackPool.registerToken reverts with TokenAlreadyRegistered whenever tokenInfo[tokenId].isActive \== true, and because AssetLendingPool.acquireDefaultedAsset re-deposits defaulted collateral into a PackMachine via depositFromPool(…) without ever clearing BuybackPool.tokenInfo[tokenId].isActive, any tokenId whose lifecycle is (won □ held by user □ used as loan collateral □ defaulted □ recycled by admin) sits in the tier pool with a stale isActive=true flag. PackVRFRouter.rawFulfillRandomWords invokes IPackMachine.fulfillRandomness with no wrapping try/catch of its own, so the revert bubbles to the Chainlink coordinator; per VRF v2.5 semantics the request is marked fulfilled, the subscription is charged, and no retry ever happens. Meanwhile _handlePayment already routed the user's USDC to financeWallet and buybackPool in the earlier openPack transaction, so the payment is unrecoverable, effectivePrizePoolSize remains decremented from the request-time reservation, and the "cursed" tokenId stays in the tier pool ready to brick the NEXT user who happens to draw it.

AFFECTED CODE

- https://github.com/nettyworth/NettyWorthV3/blob/e2490ee60c62454780fe9e86fed9d2ecfe35a954/contracts/PackMachine.sol#L377-L464
- https://github.com/nettyworth/NettyWorthV3/blob/e2490ee60c62454780fe9e86fed9d2ecfe35a954/contracts/BuybackPool.sol#L262-L285
- https://github.com/nettyworth/NettyWorthV3/blob/e2490ee60c62454780fe9e86fed9d2ecfe35a954/contracts/AssetLendingPool.sol#L487-L533
- https://github.com/nettyworth/NettyWorthV3/blob/e2490ee60c62454780fe9e86fed9d2ecfe35a954/contracts/PackVRFRouter.sol#L180-L210

BUSINESS IMPACT

Full, not refundable USDC loss per affected pack-open. Because the cursed token stays in the tier pool after the callback dies, every subsequent user who happens to draw it also loses their payment. Deterministic amplification via the normal default-lifecycle recycling path one defaulted-and-recycled card can brick an unbounded number of downstream pack purchases until admin pauses the machine and manually rescues the token.

RECOMMENDED NEXT STEP

Wrap the IBuybackPool(pool).registerToken(…) call in its own try { … } catch { … } block; on failure, push the tokenId back onto the tier pool, restore effectivePrizePoolSize, emit a CardFailed event, and continue with the rest of the pack. Alternatively, reset BuybackPool.tokenInfo[tokenId].isActive \= false inside AssetLendingPool.acquireDefaultedAsset and any admin deposit/depositFromPool path so the stale-isActive flag can never re-enter the tier pool.

RETEST

Fixed
This issue is not Fixed.

C007 · INCONSISTENT ACCOUNTING (SCWE-010)

CRITICAL

✓ FIXED

AssetNFT.batchBurn on Held-state collateral desyncs AssetLendingPool and P2PTradeEscrow accounting; loss is permanent

DESCRIPTION

AssetNFT.batchBurn allows any BURNER_ROLE holder to burn any token whose state is Held or RemovedFromPlatform. The only additional gate is the state check there is no ownership check, no "the minter can only burn its own tokens" restriction, and no exclusion for tokens currently held by protocol contracts. But Held is precisely the state used for pool-owned defaulted collateral (AssetLendingPool._initiateDefault transitions the seized token to Held) and for tokens deposited into P2PTradeEscrow (the escrow's createTrade requires the pulled AssetNFT to be in Held state and does not change the state after pull-in). A single BURNER_ROLE action that burns a token whose custody has moved to either of those contracts permanently voids the collateral without touching the holding contract's accounting: the pool's defaults[loanId].outstandingValue stays non-zero, totalDefaultedPrincipal never gets re-credited to totalDeposited, every recovery path (acquireDefaultedAsset, purchaseDefaultedAsset, rescueNFT) reverts on ERC721: invalid token ID. Eventually a lender's withdraw hits the totalDeposited totalBorrowed invariant check and reverts on underflow. On the escrow side, acceptTrade, cancelTrade, and expireTrade all revert on the burnt token's transfer, permanently freezing the trade with no admin rescue (P2PTradeEscrow has no rescueNFT).

AFFECTED CODE

- https://github.com/nettyworth/NettyWorthV3/blob/e2490ee60c62454780fe9e86fed9d2ecfe35a954/contracts/AssetNFT.sol#L284-L310
- https://github.com/nettyworth/NettyWorthV3/blob/e2490ee60c62454780fe9e86fed9d2ecfe35a954/contracts/AssetLendingPool.sol#L872-L911
- https://github.com/nettyworth/NettyWorthV3/blob/e2490ee60c62454780fe9e86fed9d2ecfe35a954/contracts/AssetLendingPool.sol#L582-L599
- https://github.com/nettyworth/NettyWorthV3/blob/e2490ee60c62454780fe9e86fed9d2ecfe35a954/contracts/P2PTradeEscrow.sol#L100-L165

BUSINESS IMPACT

Permanent pool accounting corruption on the AssetLendingPool side; the drift is monotone and eventually blocks the last lender to withdraw. On the P2P side, escrowed AssetNFTs and their counterparty-side assets are locked forever with no admin escape. Single-role-holder unrecoverable footgun.

RECOMMENDED NEXT STEP

Restrict batchBurn to RemovedFromPlatform only, and force any burn to go through a STATE_MANAGER_ROLE-gated batchSetAssetState(Held → RemovedFromPlatform) transition first. That path can enforce ownership/custody checks (e.g., disallow if ownerOf(tokenId) is a known custody contract) and cleanly separates end-of-life from live-collateral states.

RETEST

Fixed

This issue has been fixed by restricting AssetNFT.batchBurn to tokens whose state is RemovedFromPlatform the Held branch has been removed, so pool-owned defaulted collateral and P2PTradeEscrow escrowed tokens can no longer be burnt out from under the holding contracts. Retiring an idle Held token now requires a STATE_MANAGER_ROLE gated batchSetAssetState(Held → RemovedFromPlatform) first.

C008 · BUSINESS LOGIC (SC02-LOGICERRORS)

CRITICAL

✓ FIXED

BuybackPool retroactive model switch (AmountSpent □ FMV + bearer fmv=1 quote) drains pool

DESCRIPTION

TokenBuybackInfo records {pricePerCard, tier, sourcePackMachine, isActive} at registerToken time there is no model field. _executeBuyback resolves the model LIVE via _resolveModel(sourcePackMachine) on every call, meaning a token registered under AmountSpent (transparent pricePerCard basis) can be redeemed under a completely different formula if admin flips the model after registration. setPackMachineBuybackModel is gated by PACK_OPERATOR_ROLE, the same role that signs FMV quotes; initializeV2 enables both AmountSpent and FMV modes at deploy, so no additional toggle is required. _consumeFMVQuote enforces no minimum on quote.fmv a fmv=1 quote passes validation and produces payout \= 1 × buybackBps / BPS \= 0 (integer truncation). The FMV quote typehash keccak256("FMVQuote(uint256 tokenId,uint256 fmv,uint256 deadline,uint256 nonce)") does not include the seller (an inline comment even acknowledges caller; is deliberately unused), which makes the quote bearer: any relayer can broadcast a leaked operator-signed fmv=1 quote to sweep any user's NFT via a public buyback(tokenId, 0, quote, sig) call. Concrete flow: users open 1000 packs × $10 □ ~$5k accumulates in the pool at 50% buyback allocation; admin flips setPackMachineBuybackModel(machine, FMV); admin signs fmv=1 quotes for every registered token; a public relay redeems each on behalf of the holder □ holder loses NFT for 0 USDC. Admin then pause() + emergencyWithdraw() sweeps the accumulated USDC to financeWallet.

AFFECTED CODE

- https://github.com/nettyworth/NettyWorthV3/blob/e2490ee60c62454780fe9e86fed9d2ecfe35a954/contracts/BuybackPool.sol#L55-L60
- https://github.com/nettyworth/NettyWorthV3/blob/e2490ee60c62454780fe9e86fed9d2ecfe35a954/contracts/BuybackPool.sol#L262-L285
- https://github.com/nettyworth/NettyWorthV3/blob/e2490ee60c62454780fe9e86fed9d2ecfe35a954/contracts/BuybackPool.sol#L352-L359
- https://github.com/nettyworth/NettyWorthV3/blob/e2490ee60c62454780fe9e86fed9d2ecfe35a954/contracts/BuybackPool.sol#L580-L666
- https://github.com/nettyworth/NettyWorthV3/blob/e2490ee60c62454780fe9e86fed9d2ecfe35a954/contracts/BuybackPool.sol#L673-L723

BUSINESS IMPACT

Systematic drain of the BuybackPool's aggregate USDC balance AND every registered NFT held by users, without user consent, at admin's discretion. Bearer FMV quote means an operational leak of one signature enables permissionless drain against every registered token.

RECOMMENDED NEXT STEP

Snapshot the resolved BuybackModel and pricePerCard into TokenBuybackInfo at registerToken time and use those snapshots in _executeBuyback instead of live resolution. Add the seller/caller address to the FMV quote typehash so the quote binds to a specific redeemer. Enforce a minimum fmv relative to pricePerCard (e.g., fmv ≥ pricePerCard / K) to bound the discount rate.

RETEST

Fixed
This issue has been fixed by architecturally removing the dual model (AmountSpent / FMV) system from BuybackPool. The bearer FMV quote surface, model-switch admin path, and _resolveModel branching are gone; buyback payout is now uniformly appraisal × buybackBps / BPS using the on-chain AssetNFT.getAppraisalValue as the sole basis. There is no operator-signed off-chain quote path an admin can flip to drain the pool.

H001 · FRONT-RUNNING (SCWE-037)

Just-in-time lender deposit sandwich on loan repayment

HIGH

✓ FIXED

DESCRIPTION

Loan interest is fixed at origination and distributed as a single lump sum write to the Synthetix-style accInterestPerShare accumulator at repay / settleLoanRepaymentOnSale. _distributeInterest computes accInterestPerShare += (lenderPortion × PRECISION) / totalLenderDeposits against the current totalLenderDeposits, not against shares snapshotted at loan origination. An attacker watching the mempool can front-run a pending repay with lenderDeposit, capture a proportional share of the lump distribution, and lenderWithdraw in the next block. With totalLenderDeposits \= 10_000 and lenderShareBps \= 8000, a same-block lenderDeposit(10_000) ahead of a 1_000-interest repay captures 400 USDC of interest from honest lenders' expected share a 50% dilution per repay, annualised yield on the one-block capital exposure.

AFFECTED CODE

- https://github.com/nettyworth/NettyWorthV3/blob/e2490ee60c62454780fe9e86fed9d2ecfe35a954/contracts/AssetLendingPool.sol#L153-L198
- https://github.com/nettyworth/NettyWorthV3/blob/e2490ee60c62454780fe9e86fed9d2ecfe35a954/contracts/AssetLendingPool.sol#L214-L258
- https://github.com/nettyworth/NettyWorthV3/blob/e2490ee60c62454780fe9e86fed9d2ecfe35a954/contracts/AssetLendingPool.sol#L346-L363
- https://github.com/nettyworth/NettyWorthV3/blob/e2490ee60c62454780fe9e86fed9d2ecfe35a954/contracts/AssetLendingPool.sol#L913-L926

BUSINESS IMPACT

Direct wealth transfer from honest lenders to attacker per repay; when totalLenderDeposits \== 0 at the moment of attack, attacker captures 100% of the lender portion. Compounds with every loan repayment.

RECOMMENDED NEXT STEP

Add a deposit cooldown that excludes fresh capital from interest claims until ~1 day after deposit; OR snapshot (lenderDeposits, totalLenderDeposits) at _originateLoan and have _distributeInterest read from the per-loan snapshot.

RETEST

Fixed
By snapshotting lenderDepositsSnapshot \= totalLenderDeposits per-loan at _originateLoan and passing it into _distributeInterest at repay and default recovery, fresh JIT deposits no longer dilute the honest lenders' share of that loan's interest.

H002 · SIGNATURE REPLAY ATTACKS (SCWE-055)

FMV quote is a bearer instrument signer-binding never finished

HIGH

✓ FIXED

DESCRIPTION

BuybackPool.FMV_QUOTE_TYPEHASH binds only (tokenId, fmv, deadline, nonce) there is no seller address in the signed struct. _consumeFMVQuote receives a caller argument but explicitly discards it (caller; no-op) with a code comment marking it as deferred-design. Any account currently holding the NFT can redeem any leaked or stolen quote before its deadline. The intended workflow ("operator quotes a specific holder; that holder redeems") is structurally unenforced; any leaked quote becomes a free-money instrument for the next person to acquire the NFT (cheaply, on the secondary market) and call buyback with the leaked sig.

AFFECTED CODE

- https://github.com/nettyworth/NettyWorthV3/blob/e2490ee60c62454780fe9e86fed9d2ecfe35a954/contracts/BuybackPool.sol#L105-L107
- https://github.com/nettyworth/NettyWorthV3/blob/e2490ee60c62454780fe9e86fed9d2ecfe35a954/contracts/BuybackPool.sol#L669-L719

BUSINESS IMPACT

Per leaked quote: fmv × bBps − market_price_of_NFT drained from BuybackPool. Operator-key compromise is catastrophic attacker mints unbounded quotes.

RECOMMENDED NEXT STEP

It is recommended to add address seller to the typehash and enforce quote.seller \== caller in _consumeFMVQuote.

RETEST

Fixed
This issue has been fixed by adding address seller to FMV_QUOTE_TYPEHASH and enforcing quote.seller \== caller in _consumeFMVQuote (revert BuybackPool__FMVQuoteSellerMismatch) a leaked quote can no longer be redeemed by anyone other than the intended seller.

H003 · UNCAUGHT EXCEPTIONS (SCWE-004)

Bundle loan over-release marketplace sale of one token releases the entire collateral bundle

HIGH

✓ FIXED

DESCRIPTION

When a marketplace sale's tokenId is collateralised in a multi-NFT bundle loan, _executeSale calls settleLoanRepaymentOnSale(loanId, ...), which iterates loan.tokenIds and transfers every NFT in the bundle to the buyer not just the listed token. The marketplace's pricing check (gross ≥ collectibleFee + royalty + loanDebt) only verifies the buyer pays off the full loan, but it doesn't verify the sold token represents the full collateral. A borrower with a bundle loan [X, Y, Z] who lists only token X at loanDebt + small_margin causes the pool to silently deliver [X, Y, Z] to the buyer; the seller pays off their loan but loses N-1 extra NFTs for ~0 net proceeds. This is also a money laundering vector: bundle high value NFTs, list one cheap, have an accomplice "buy" and transfer the whole bundle through the marketplace.

AFFECTED CODE

- https://github.com/nettyworth/NettyWorthV3/blob/e2490ee60c62454780fe9e86fed9d2ecfe35a954/contracts/NettyWorthMarketplace.sol#L474-L558
- https://github.com/nettyworth/NettyWorthV3/blob/e2490ee60c62454780fe9e86fed9d2ecfe35a954/contracts/AssetLendingPool.sol#L214-L258

BUSINESS IMPACT

Silent over-delivery of NFTs to the buyer per single-token sale of bundle collateral. Severe seller harm; laundering vector with hidden economics for any colluding pair.

RECOMMENDED NEXT STEP

In marketplace _executeSale, query the loan's token list and reject single-token sales of bundle loans.

RETEST

Fixed

The issue is fixed by calling getLoanCollateralCount(activeLoanId) and reverting with Marketplace__CannotPartiallySettleBundleLoan when the count is greater than 1 in _settleSale single-token sales of bundle loans are rejected before over-releasing the collateral.

H004 · FRONT-RUNNING (SCWE-037)

buyWithSignature SignedListing is a bearer instrument front-running steals private deals and griefs intended buyers

HIGH

✓ FIXED

DESCRIPTION

SIGNED_LISTING_TYPEHASH binds (seller, collection, tokenId, paymentToken, price, nonce, expiry) but does not include a buyer address. The signature is therefore a bearer instrument; anyone who submits it first is the buyer (buyWithSignature sets buyer \= msg.sender). Any mempool watcher can extract (listing, sig) from a pending honest buyer transaction, copy the calldata with higher gas, and become the buyer themselves; the victim's transaction reverts with Marketplace__NonceUsed. The most-impactful case is off-chain negotiated private sales: a seller signs at a below market price for a specific counterparty and shares the signature privately; the buyer's submission inevitably broadcasts the full listing struct (including the below market price) in calldata, and MEV bots scanning for buyWithSignature calls where listing.price is significantly below market value identify and snipe the listing immediately. The intended buyer loses gas + the negotiated discount; the attacker captures (marketPrice − listingPrice) per snipe.

AFFECTED CODE

- https://github.com/nettyworth/NettyWorthV3/blob/e2490ee60c62454780fe9e86fed9d2ecfe35a954/contracts/NettyWorthMarketplace.sol#L63-L66
- https://github.com/nettyworth/NettyWorthV3/blob/e2490ee60c62454780fe9e86fed9d2ecfe35a954/contracts/NettyWorthMarketplace.sol#L169-L215

BUSINESS IMPACT

Direct economic transfer from intended buyer to attacker per snipe, equal to the negotiated discount. For privately shared below market listings the gap can be substantial; for fair-market listings the result is generic buyer-side griefing (gas burn + denied NFT acquisition).

RECOMMENDED NEXT STEP

Add buyer to the typehash with address(0) meaning "open listing" .

RETEST

Fixed
This issue has been fixed by adding a buyer field to SIGNED_LISTING_TYPEHASH and the SignedListing struct, and enforcing listing.buyer == recipient when non-zero in _validateListing private listings are now bound to a specific buyer, with address(0) meaning open.

H005 · UNCAUGHT EXCEPTIONS (SCWE-004)

HIGH

Cross-Collection tokenId Collision latent collateral heist activates on multi-collection allowlist

✓ FIXED

DESCRIPTION

The marketplace's loan-aware settlement queries the lending pool using only tokenId (no collection). The pool's tokenIdToActiveLoan mapping is similarly keyed on uint256 alone, implicitly assuming all loaned tokenIds are AssetNFTs. The marketplace is explicitly designed as multi-collection (allowedCollections mapping, setAllowedCollection admin setter, code comment: "multi-collection future-proofing"). The instant admin allowlists ANY second collection (the documented expansion path no malice required), an attacker who controls a tokenId in the new collection that collides with an active AssetNFT loan tokenId can list it on the marketplace; _executeSale sees a non-zero loanDebt for that tokenId (false positive the loan is for AssetNFT), enters the loan branch which skips the direct collection transfer, and instructs the pool to deliver its AssetNFT collateral to the buyer. The seller keeps the worthless listed NFT; the lending-pool collateral owner (a third party) silently loses their AssetNFT. Each contract's own books balance the violation is the cross-contract identity mismatch, invisible to on-chain anomaly detection.

AFFECTED CODE

- https://github.com/nettyworth/NettyWorthV3/blob/e2490ee60c62454780fe9e86fed9d2ecfe35a954/contracts/NettyWorthMarketplace.sol#L500-L501
- https://github.com/nettyworth/NettyWorthV3/blob/e2490ee60c62454780fe9e86fed9d2ecfe35a954/contracts/NettyWorthMarketplace.sol#L531-L539
- https://github.com/nettyworth/NettyWorthV3/blob/e2490ee60c62454780fe9e86fed9d2ecfe35a954/contracts/AssetLendingPoolConfig.sol#L50
- https://github.com/nettyworth/NettyWorthV3/blob/e2490ee60c62454780fe9e86fed9d2ecfe35a954/contracts/AssetLendingPoolConfig.sol#L440-L455
- https://github.com/nettyworth/NettyWorthV3/blob/e2490ee60c62454780fe9e86fed9d2ecfe35a954/contracts/NettyWorthMarketplace.sol#L408-L414

BUSINESS IMPACT

Direct theft of lending-pool collateral from third-party loan holders. Per attack: ≈ (NFT market value − fees). Permanent; ERC-721 transfer is final. Scales linearly with (active loans × allowlisted collections × colliding tokenIds).

RECOMMENDED NEXT STEP

It is recommended to re-key tokenIdToActiveLoan to (collection, tokenId) for true multi-collection lending. Fix before allowlisting any second collection.

RETEST

Fixed
By gating the pool debt lookup with collection \== lendingPool.getAssetNFT() in _settleSale before querying tokenIdToActiveLoan a second whitelisted collection with a colliding tokenId can no longer trigger the loan branch. (The pool-side mapping remains tokenId-keyed, but the collision surface is closed at the marketplace call site.)

H006 · BUSINESS LOGIC (SC02-LOGICERRORS)

HIGH

✓ FIXED

BuybackPool._executeBuyback FMV-quote nonce not reset on registerToken; leaked stale quote drains pool after model flip

DESCRIPTION

registerToken overwrites $.tokenInfo[tokenId] fully but never touches $.fmvQuoteNonce[tokenId]. _consumeFMVQuote (the only writer of fmvQuoteNonce) is invoked only when the resolved model is FMV every AmountSpent-lifecycle buyback leaves the nonce untouched. Attack: (a) operator signs FMVQuote(tokenId=T, fmv=10e12, deadline=+365d, nonce=0) while the machine's model is AmountSpent; (b) the token's current holder redeems T under AmountSpent payout uses pricePerCard, nonce stays at 0; (c) T is redeposited to the tier pool and later re-won or purchased on the secondary market by an attacker; registerToken(T, …) succeeds because isActive was cleared in step (b) but fmvQuoteNonce[T] is still 0; (d) admin flips the machine to FMV; (e) attacker calls buyback(T, bytes32(0), staleQuote, staleSig) deadline still valid, signer still holds PACK_OPERATOR_ROLE, quote nonce (0) matches storage nonce (0), quote passes basis \= 10e12, payout \= 10e12 × 8000 / 10000 \= 8e12 USDC. Pool drained. Independent of C008: even if the model-switch itself were trusted, any historically-leaked FMV quote with a distant deadline is a permanent liability against the pool's balance.

AFFECTED CODE

- https://github.com/nettyworth/NettyWorthV3/blob/e2490ee60c62454780fe9e86fed9d2ecfe35a954/contracts/BuybackPool.sol#L262-L285
- https://github.com/nettyworth/NettyWorthV3/blob/e2490ee60c62454780fe9e86fed9d2ecfe35a954/contracts/BuybackPool.sol#L580-L666
- https://github.com/nettyworth/NettyWorthV3/blob/e2490ee60c62454780fe9e86fed9d2ecfe35a954/contracts/BuybackPool.sol#L673-L723

BUSINESS IMPACT

Full pool drain via any leaked FMV quote whose deadline outlives the token's redeposit-and-model-flip cycle. Amplified by the bearer nature of FMV quotes. a single leaked signature is exploitable by any relayer against any subsequent owner of the target tokenId.

RECOMMENDED NEXT STEP

Bump (or reset to zero) $.fmvQuoteNonce[tokenId] inside registerToken so every re-registration invalidates all prior quotes. Additionally embed a per-lifecycle epoch counter into the FMV typehash so a quote binds to a specific (tokenId, lifecycle) tuple.

RETEST

Fixed
This issue has been fixed by architecturally removing FMV quotes from BuybackPool. The FMV_QUOTE_TYPEHASH, per-token fmvQuoteNonce mapping, and _consumeFMVQuote path have all been deleted; buyback pricing is derived exclusively from the on-chain appraisal so no leaked quote can survive a re-registration cycle to drain the pool.

H007 · BUSINESS LOGIC (SC02-LOGICERRORS)

HIGH

AssetNFT.initiateShipment no maxFee slippage; admin setAppraisal front-run drains user's USDC approval

✓ FIXED

DESCRIPTION

initiateShipment reads the live appraisal via IAssetLendingPool(lendingPool).getAppraisal(tokenId).value and computes fee \= value × redemptionFeeBps / BPS, then pulls the fee via safeTransferFrom(caller, treasury, fee). There is no user-supplied maxFee, no expected-appraisal parameter, no signature. AssetLendingPoolConfig.setAppraisal is gated only by onlyOwner, no timelock, no per-tick delta cap. FeeController.MAX_REDEMPTION_FEE \= 10_000 bps (100% validated in scan), and treasury is admin-set. So an owner mempool-front-run of setAppraisal(tokenId, HUGE) right before a user's initiateShipment(tokenId) tx causes the user's tx to pull HUGE × redemptionFeeBps / BPS from their USDC balance to the admin-controlled treasury. Under typical UX (unlimited USDC approval to the AssetNFT contract), the entire USDC balance is drainable in one race. Concrete: NFT appraised at $500 □ user expects $25 fee. Admin front-runs setAppraisal(tokenId, 20_000_000e6). User's tx pulls 20_000_000e6 × 500 / 10_000 \= 1_000_000e6 \= $1M (capped by user's balance). This is a direct value-extraction primitive, not a griefing bug.

AFFECTED CODE

- https://github.com/nettyworth/NettyWorthV3/blob/e2490ee60c62454780fe9e86fed9d2ecfe35a954/contracts/AssetNFT.sol#L504-L545
- https://github.com/nettyworth/NettyWorthV3/blob/e2490ee60c62454780fe9e86fed9d2ecfe35a954/contracts/AssetLendingPoolConfig.sol#L184-L191
- https://github.com/nettyworth/NettyWorthV3/blob/e2490ee60c62454780fe9e86fed9d2ecfe35a954/contracts/FeeController.sol#L20-L60

BUSINESS IMPACT

Direct theft of the user's USDC allowance, bounded only by their approval and the fee cap (100%). Attack window is the mempool interval between initiateShipment broadcast and inclusion, weaponisable by any owner-key holder.

RECOMMENDED NEXT STEP

Add uint256 maxFee (or uint256 expectedAppraisalValue) parameter to initiateShipment; revert if the computed fee exceeds the user's supplied bound. Alternatively require the caller to pass a signed appraisal snapshot the pool countersigns.

RETEST

Fixed
This issue has been fixed by adding a uint256 maxFee parameter to AssetNFT.initiateShipment and reverting with AssetNFT__FeeExceedsMax(fee, maxFee) when the computed fee (appraisal × redemptionFeeBps / BPS) exceeds the caller-supplied bound users can now front-run-proof themselves against an admin setAppraisal race by supplying a tight upper bound (or type(uint256).max to accept any fee).

H008 · BUSINESS LOGIC (SC02-LOGICERRORS)

HIGH

✓ FIXED

AssetLendingPool._collectOriginationFee live-read at disbursement; admin setOriginationFee front-run shrinks borrower's disbursement to zero

DESCRIPTION

_collectOriginationFee reads live $.originationFeeBps and $.feeWallet at disbursement time. It is called from _borrow AFTER _originateLoan meaning the NFT is already locked, the Loan struct is already written, and totalBorrowed/activeLoanCount are already committed by the time the fee is calculated. AssetLendingPoolConfig.setOriginationFee accepts bps ≤ BPS \= 10_000 (100%) with no timelock and no per-block rate-limit. _borrow and borrowBundle take no maxFeeBps slippage parameter. Concrete: borrower expects 1% fee on 10,000 USDC 9,900 net. Owner front-runs setOriginationFee(5000, ownerWallet); borrower's tx executes, receives 5,000 USDC net and still owes 10,000 principal plus interest computed on the FULL 10,000. The same defect exists in financeMarketplacePurchase (buyer pays the origination fee to feeWallet, drainable up to the buyer's approval to the pool).

AFFECTED CODE

- https://github.com/nettyworth/NettyWorthV3/blob/e2490ee60c62454780fe9e86fed9d2ecfe35a954/contracts/AssetLendingPool.sol#L739-L747
- https://github.com/nettyworth/NettyWorthV3/blob/e2490ee60c62454780fe9e86fed9d2ecfe35a954/contracts/AssetLendingPool.sol#L751-L785
- https://github.com/nettyworth/NettyWorthV3/blob/e2490ee60c62454780fe9e86fed9d2ecfe35a954/contracts/AssetLendingPool.sol#L272-L337
- https://github.com/nettyworth/NettyWorthV3/blob/e2490ee60c62454780fe9e86fed9d2ecfe35a954/contracts/AssetLendingPoolConfig.sol#L302-L313

BUSINESS IMPACT

The owner extracts up to 100% of the requested principal from any in-flight borrow or finance-purchase transaction. Borrower/buyer is left carrying interest on principal they never received. Same class as Dev M003 (retroactive lenderShareBps) but on the borrower-side fee. Direct value-extraction primitive via mempool race.

RECOMMENDED NEXT STEP

Add a uint256 maxFeeBps parameter to borrow, borrowBundle, and financeMarketplacePurchase and revert when the live fee exceeds it. Alternatively snapshot originationFeeBps into the Loan struct at origination.

RETEST

Fixed
This issue has been fixed by snapshotting originationFeeBps into Loan.originationFeeBpsSnapshot at _originateLoan and having _collectOriginationFee read the per-loan snapshot in both _borrow and financeMarketplacePurchase a mid-flight setOriginationFee call can no longer retroactively enlarge the fee taken from an in-flight borrower.

H009 · ACCESS CONTROL (SCWE-016)

HIGH

AssetLendingPool.financeMarketplacePurchase skips listing.buyer enforcement private listing hijack

✓ FIXED

DESCRIPTION

The C003 fix ported the marketplace's SIGNED_LISTING_TYPEHASH into AssetLendingPool verbatim including the H004 buyer field so signatures over private (buyer-bound) listings are cryptographically distinguishable. The pool builds the struct hash with listing.buyer at L285-297, verifies the signature against the marketplace's EIP-712 domain at L276-303, and the same signature therefore verifies at both contracts. But financeMarketplacePurchase never enforces listing.buyer against msg.sender the way NettyWorthMarketplace._validateListing does at L277-278 (if (listing.buyer != address(0) && listing.buyer != recipient) revert Marketplace__NotIntendedBuyer). A seller who signs a private listing intended for Alice via the marketplace-financed flow a flow that structurally requires seller to have granted an ERC-721 approval to the pool for the tokenId (L347 assetNFT.transferFrom(listing.seller, address(this), tokenId)) can have that signature hijacked by any mempool observer. Attacker Bob calls financeMarketplacePurchase(listing, sig, minDeposit, term) with msg.sender \= Bob; signature verifies, the buyer field is ignored, Bob becomes the borrower, the NFT is collateralised for Bob's loan, and seller receives listing.price. Alice is defrauded of the private allocation. Additionally the pool uses a separate financeNonces mapping (L94) from the marketplace's usedNonces (L105), and the pool exposes no cancelNonce equivalent so a seller who becomes aware of a leaked signature has no on-chain way to revoke it at the pool, even after calling marketplace.cancelNonce(N). Same H004-class defect surviving at the pool boundary.

AFFECTED CODE

- https://github.com/nettyworth/NettyWorthV3/blob/d1475c0f34c73b4f1cb4ffee995e48a8d38d0df2/contracts/AssetLendingPool.sol#L253-L256
- https://github.com/nettyworth/NettyWorthV3/blob/d1475c0f34c73b4f1cb4ffee995e48a8d38d0df2/contracts/AssetLendingPool.sol#L259-L382
- https://github.com/nettyworth/NettyWorthV3/blob/d1475c0f34c73b4f1cb4ffee995e48a8d38d0df2/contracts/AssetLendingPool.sol#L285-L303
- https://github.com/nettyworth/NettyWorthV3/blob/d1475c0f34c73b4f1cb4ffee995e48a8d38d0df2/contracts/AssetLendingPool.sol#L94
- https://github.com/nettyworth/NettyWorthV3/blob/d1475c0f34c73b4f1cb4ffee995e48a8d38d0df2/contracts/NettyWorthMarketplace.sol#L238-L279

BUSINESS IMPACT

Direct value extraction from any seller who uses the intended marketplace-financed flow: attacker captures (market price − listing.price) whenever a private listing is set below market the classic private-discount steal. The intended-buyer field is present but non-authoritative on the pool side, so H004's guarantee (private listings bound to a specific buyer) does not extend to financeMarketplacePurchase. Compounded by the split-nonce namespace: seller cannot revoke a leaked signature at the pool even after cancelling it at the marketplace, so any leaked signature remains a live liability against every seller-approved pool tokenId until expiry. No admin role required, no special access single-tx mempool front-run.

RECOMMENDED NEXT STEP

Add a buyer enforcement check inside financeMarketplacePurchase after the signature is verified, mirroring NettyWorthMarketplace._validateListing:

```
if (listing.buyer != address(0) && listing.buyer != msg.sender) revert AssetLendingPool__NotIntendedBuyer();
```

RETEST

Fixed

This issue has been fixed by adding the recommended buyer enforcement check inside financeMarketplacePurchase in AssetLendingPool.sol: if (listing.buyer != address(0) && listing.buyer != msg.sender) revert AssetLendingPool__NotIntendedBuyer();

H010 · ACCESS CONTROL (SCWE-016)

HIGH

FIXED

Seller nonce cancellation asymmetry marketplace.cancelNonce burns only usedNonces, leaving pool.financeNonces redeemable

DESCRIPTION

The C003 remediation (commit 9422875) added AssetLendingPool.financeMarketplacePurchase which verifies a marketplace-domain-signed SignedListing against the pool's own financeNonces mapping (AssetLendingPool.sol:94, marked at :321-324). NettyWorthMarketplace.cancelNonce (NettyWorthMarketplace.sol:690-696) burns only usedNonces (NettyWorthMarketplace.sol:105). Both contracts verify the identical SIGNED_LISTING_TYPEHASH under the marketplace EIP-712 domain (pool reconstructs the domain separator explicitly at AssetLendingPool.sol:276-284), so the same signature is valid at both entry points a fact independent of the H009 buyer-binding fix in commit 7b954da. When a seller signs an open listing (buyer=0x0), the signature leaks (Discord, relayer, off-chain marketplace UI, etc.), and the seller calls marketplace.cancelNonce(N) believing this revokes the signature the pool's financeNonces[seller][N] is unaffected. An attacker can then call financeMarketplacePurchase(listing, sig, deposit, term); the signature verifies at AssetLendingPool.sol:301, the pool pulls the NFT via transferFrom(listing.seller, address(this), tokenId) at :351, and the attacker becomes the borrower on a collateralised loan against the seller's NFT a listing the seller had explicitly repudiated. The pool exposes no cancelFinanceNonce function; the only pool-side sellerinvalidation path is either (a) letting listing.expiry pass or (b) revoking the ERC-721 approval to the pool, both of which are indirect and slow.

AFFECTED CODE

- https://github.com/nettyworth/NettyWorthV3/blob/7b954dae98a99a2399dcef315203b7add7df6557/contracts/AssetLendingPool.sol#L94
- https://github.com/nettyworth/NettyWorthV3/blob/7b954dae98a99a2399dcef315203b7add7df6557/contracts/AssetLendingPool.sol#L259-L390
- https://github.com/nettyworth/NettyWorthV3/blob/7b954dae98a99a2399dcef315203b7add7df6557/contracts/NettyWorthMarketplace.sol#L105
- https://github.com/nettyworth/NettyWorthV3/blob/7b954dae98a99a2399dcef315203b7add7df6557/contracts/NettyWorthMarketplace.sol#L690-L696

BUSINESS IMPACT

Sellers cannot invalidate a leaked signature at the pool. Any leaked open-listing signature remains a live liability against every seller-approved pool tokenId until listing.expiry. Seller loses full NFT value; attacker captures (marketValue − listing.price) if listing.price is below market, or the entire NFT collateral if listing.price is low. No admin role required anyone with the leaked signature can execute the attack in a single mempool observation. The same signature pathway that H009 addressed via buyer-binding remains open here via the cancellation-namespace split. Effectively re-opens the C005-class revocation gap at a different contract boundary.

RECOMMENDED NEXT STEP

Preferred: unify the nonce namespace. Have financeMarketplacePurchase delegate consumption to marketplace via a new INettyWorthMarketplace.consumeListingNonce(seller, nonce) call, gated on the pool holding a dedicated LENDING_POOL_ROLE at the marketplace. This makes cancelNonce authoritative across both contracts and eliminates the possibility of state divergence. Fallback: add AssetLendingPool.cancelFinanceNonce(uint256 nonce):

```
function cancelFinanceNonce(uint256 nonce) external {
    _getStorage().financeNonces[msg.sender][nonce] = true;
    emit FinanceNonceCancelled(msg.sender, nonce);
}
```

And update UI/docs so sellers know to cancel both contracts. The fallback leaves a UX cliff; the preferred fix removes the cliff entirely.

RETEST

Fixed

This issue has been fixed in commit 48934e6 by adopting the fallback remediation the pool now exposes cancelFinanceNonce(uint256 nonce) in AssetLendingPool.sol

M001 · UNCAUGHT EXCEPTIONS (SCWE-004)

MEDIUM

Stranded pack open VRF callback can fail and consume payment without delivering cards or refund

✓ FIXED

DESCRIPTION

PackMachine.fulfillRandomness performs the prize-NFT transfer and BuybackPool.registerToken inside the same try block. If registerToken reverts (BuybackPool paused, blacklist, validator rejection), the entire try block reverts atomically including the preceding _swapAndPopTier storage writes. The catch then unconditionally pushes the tokenId back into the tier pool, producing a duplicate entry: once because the swap and pop's storage writes were reverted, once because catch pushed it again. effectivePrizePoolSize is over-counted by 1 per occurrence. Subsequent opens may select the duplicate index, find the NFT gone (it was actually transferred in a different open), revert at transferFrom, catch and restore it again an unbounded growth loop. Additionally pendingOpens[requestId] is deleted before the percard loop, so there is no retry path for partially-failed callbacks; the user's payment + promo redemption are consumed regardless.

AFFECTED CODE

- https://github.com/nettyworth/NettyWorthV3/blob/e2490ee60c62454780fe9e86fed9d2ecfe35a954/contracts/PackMachine.sol#L377-L462
- https://github.com/nettyworth/NettyWorthV3/blob/e2490ee60c62454780fe9e86fed9d2ecfe35a954/contracts/PackVRFRouter.sol#L175-L192

BUSINESS IMPACT

User payment consumed without full delivery, promo discount permanently burned with no unredeem, effectivePrizePoolSize drifts above real capacity.

RECOMMENDED NEXT STEP

Separate the transfer from the registration so registerToken's revert can't rollback the swap-and-pop or trigger duplicate push

RETEST

Fixed
By performing the ERC721 transfer inside the outer try/catch and calling registerToken only after a successful transfer inside its own dedicated inner try/catch a registerToken revert can no longer rollback the swap-and-pop or trigger the duplicate-push restore branch.

M002 · INCONSISTENT ACCOUNTING (SCWE-010)

MEDIUM

borrowerLoans array is append-only unbounded growth, no closing-path cleanup

✓ FIXED

DESCRIPTION

The _originateLoan pushes loanId into borrowerLoans[borrower]. No closing path (repay, settleLoanRepaymentOnSale, _initiateDefault, acquireDefaultedAsset) removes the entry. getBorrowerLoans returns the borrower's lifetime history with no in-array distinguisher between active, paid, and defaulted loans. Active borrowers accumulate hundreds of entries; off-chain integrators querying the view function eventually hit RPC response-size limits, and any future on-chain consumer iterating the array hits the block gas limit. The asymmetry is deliberate-looking only on the write side; other loan-state tracking (tokenIdToActiveLoan, activeLoanCount, loan.isPaid) is cleaned correctly.

AFFECTED CODE

- https://github.com/nettyworth/NettyWorthV3/blob/e2490ee60c62454780fe9e86fed9d2ecfe35a954/contracts/AssetLendingPool.sol#L836
- https://github.com/nettyworth/NettyWorthV3/blob/e2490ee60c62454780fe9e86fed9d2ecfe35a954/contracts/AssetLendingPool.sol#L153-L198
- https://github.com/nettyworth/NettyWorthV3/blob/e2490ee60c62454780fe9e86fed9d2ecfe35a954/contracts/AssetLendingPool.sol#L871-L910
- https://github.com/nettyworth/NettyWorthV3/blob/e2490ee60c62454780fe9e86fed9d2ecfe35a954/contracts/AssetLendingPool.sol#L624-L628

BUSINESS IMPACT

View function breaks at scale; storage bloat is permanent. Heavy DeFi users can grief their own address by repeatedly borrowing/repaying small loans.

RECOMMENDED NEXT STEP

Swap-and-pop on close with a per-loan index. Maintain mapping(uint256 loanId \=> uint256 index) borrowerLoanIndex, populate on _originateLoan, and add a _removeBorrowerLoan helper called from all four close paths.

RETEST

Fixed
This vulnerability has been fixed by adding a borrowerLoanIndex mapping and a _removeBorrowerLoan swap-and-pop helper invoked from _settleLoanRepayment (repay + settleLoanRepaymentOnSale) and _initiateDefault the borrowerLoans array now shrinks on every close path, bounding storage growth.

M003 · UNCAUGHT EXCEPTIONS (SCWE-004)

Mutable lenderShareBps retroactively reprices in-flight loan interest

MEDIUM

✓ FIXED

DESCRIPTION

The _distributeInterest reads $.lenderShareBps at distribution time, not at loan origination. The owner can change it mid-loan via setLenderConfig (no two-step, no timelock). A lender who deposited expecting 80% interest share can be diluted to 10% on the next repay; conversely, the owner can sandwich a repay (set low, repay, set high) to capture more protocol fee. Even unintentionally, an honest rate adjustment retroactively dilutes positions already accrued under the old rate. There is no per-loan snapshot of the share parameter.

AFFECTED CODE

- https://github.com/nettyworth/NettyWorthV3/blob/e2490ee60c62454780fe9e86fed9d2ecfe35a954/contracts/AssetLendingPoolConfig.sol#L332-L341
- https://github.com/nettyworth/NettyWorthV3/blob/e2490ee60c62454780fe9e86fed9d2ecfe35a954/contracts/AssetLendingPool.sol#L913-L926

BUSINESS IMPACT

Systematic deprivation of lender expected interest at owner discretion.owner has powerful tools to manipulate lender returns.

RECOMMENDED NEXT STEP

Snapshot lenderShareBps perloan at _originateLoan and read the snapshot in _distributeInterest.

RETEST

Fixed
This issue has been fixed by snapshotting lenderShareBpsSnapshot per-loan at _originateLoan and having _distributeInterest read the snapshot from the Loan struct rather than live config mid-loan setLenderConfig changes no longer reprice interest on in-flight loans.

M004 · MISSING FUNCTIONALITY (SCWE-006)

MEDIUM

PackMachine config setters not pause-gated race window between user payment and VRF callback

✓ FIXED

DESCRIPTION

setPrice requires the contract to be paused. The other PackMachine config setters (setTierWeights, setBuybackAllocation, setRetentionThreshold, setBuybackPool, setAuthorizedDepositor) do NOT. While a user has payment in flight (after _handlePayment / Permit2 but before VRF callback fires minutes on mainnet), the operator can change probability weights, buyback split, or pool routing. The user receives prizes computed under post-payment parameters they never agreed to. setBuybackAllocation(0) mid-flight means the user paid the buyback share but no USDC actually goes to the pool. The asymmetry between setPrice and the others looks like an oversight rather than a design choice.

AFFECTED CODE

- https://github.com/nettyworth/NettyWorthV3/blob/e2490ee60c62454780fe9e86fed9d2ecfe35a954/contracts/PackMachine.sol#L582-L593
- https://github.com/nettyworth/NettyWorthV3/blob/e2490ee60c62454780fe9e86fed9d2ecfe35a954/contracts/PackMachine.sol#L625-L632
- https://github.com/nettyworth/NettyWorthV3/blob/e2490ee60c62454780fe9e86fed9d2ecfe35a954/contracts/PackMachine.sol#L595-L602
- https://github.com/nettyworth/NettyWorthV3/blob/e2490ee60c62454780fe9e86fed9d2ecfe35a954/contracts/PackMachine.sol#L636-L643

BUSINESS IMPACT

Probability distributions advertised at payment time differ from those at fulfillment; pool funding can be silently halted.

RECOMMENDED NEXT STEP

Add if (!paused()) revert PackMachine__NotPaused(); to all five config setters consistent with setPrice. Alternative: snapshot the params in pendingOpens so the open is fulfilled at the params active at payment time.

RETEST

Fixed
This vulnerability is fixed via the snapshot alternative. PendingOpen now stores tierWeightsSnapshot[6] and buybackAllocationBpsSnapshot captured at _requestVRF time, and fulfillRandomness reads the snapshots instead of live PackRegistry values, so a mid-flight PackRegistry setter (setPackTierWeights / setPackBuybackAllocation / setPackPrice) cannot alter the distribution or buyback routing of an in-flight open. In addition, PackMachine.setBuybackPool and setAuthorizedDepositor are now pause-gated with zero-address checks.

M005 · MISSING FUNCTIONALITY (SCWE-006)

MEDIUM

Marketplace allowlist setter has no compatibility canary fee-on-transfer token addition will brick settlement

✓ FIXED

DESCRIPTION

setAllowedPaymentToken(token, true) performs only address/bool validation. The marketplace accepts signed listings denominated in the allowlisted token, but _executeSale pays out gross \= collectibleFee + royalty + loanDebt + sellerProceeds and assumes safeTransferFrom(buyer, this, gross) delivers exactly gross units. If the allowlisted token is fee-on-transfer (FoT), rebasing-down, paused, blacklist-active against the marketplace, or non-standard return-value, the marketplace receives less than gross (or the transfer reverts), and every settlement reverts. The misconfiguration is silent until the first failed sale, stranding any listings already created in that token.

Under the current deployment only USDC is allowlisted. USDC has no fee-on-transfer, no rebasing, returns boolean on all transfers, and never silently shorts the recipient so the failure mode is dormant today. However, the contract is architecturally multi-token (the allowedPaymentTokens mapping and setAllowedPaymentToken setter exist for future expansion). The moment admin allowlists any fee-on-transfer token (e.g., USDT once its admin-settable fee switch is turned on, PAXG, certain reflection tokens), safeTransferFrom(buyer, this, gross) delivers gross fee to the marketplace while the payout math still references gross, so the downstream payouts (collectibleFee + royalty + loanDebt + sellerProceeds) overdraw and every settlement in that token reverts. Listings created in the new token would be stranded until the admin removes the allowlist. The setter has no compatibility canary today admin's allowlist decision is the only safeguard.

AFFECTED CODE

- https://github.com/nettyworth/NettyWorthV3/blob/e2490ee60c62454780fe9e86fed9d2ecfe35a954/contracts/NettyWorthMarketplace.sol#L417-L422
- https://github.com/nettyworth/NettyWorthV3/blob/e2490ee60c62454780fe9e86fed9d2ecfe35a954/contracts/NettyWorthMarketplace.sol#L518
- https://github.com/nettyworth/NettyWorthV3/blob/e2490ee60c62454780fe9e86fed9d2ecfe35a954/contracts/NettyWorthMarketplace.sol#L484
- https://github.com/nettyworth/NettyWorthV3/blob/e2490ee60c62454780fe9e86fed9d2ecfe35a954/contracts/NettyWorthMarketplace.sol#L489-L497
- https://github.com/nettyworth/NettyWorthV3/blob/e2490ee60c62454780fe9e86fed9d2ecfe35a954/contracts/NettyWorthMarketplace.sol#L515
- https://github.com/nettyworth/NettyWorthV3/blob/e2490ee60c62454780fe9e86fed9d2ecfe35a954/contracts/NettyWorthMarketplace.sol#L521-L523
- https://github.com/nettyworth/NettyWorthV3/blob/e2490ee60c62454780fe9e86fed9d2ecfe35a954/contracts/NettyWorthMarketplace.sol#L526-L528
- https://github.com/nettyworth/NettyWorthV3/blob/e2490ee60c62454780fe9e86fed9d2ecfe35a954/contracts/NettyWorthMarketplace.sol#L534-L535
- https://github.com/nettyworth/NettyWorthV3/blob/e2490ee60c62454780fe9e86fed9d2ecfe35a954/contracts/NettyWorthMarketplace.sol#L542-L544
- https://github.com/nettyworth/NettyWorthV3/blob/e2490ee60c62454780fe9e86fed9d2ecfe35a954/contracts/NettyWorthMarketplace.sol#L546-L557
- https://github.com/nettyworth/NettyWorthV3/blob/e2490ee60c62454780fe9e86fed9d2ecfe35a954/contracts/AssetLendingPool.sol#L214-L258

BUSINESS IMPACT

Under USDC-only deployment: no live impact. The moment admin allowlists any fee-on-transfer / rebasing / non-standard ERC-20: every settlement in that token reverts new listings created normally but settled nowhere; wasted gas + UX broken for any allowlisted weird token; existing token-denominated listings stranded. Must be fixed before any allowlist extension; otherwise the foot-gun is one admin call away from production breakage.

RECOMMENDED NEXT STEP

It is recommended to implement before -after balance calculation to fix fee on transfer issue. Add a one-time compatibility canary in the setter that catches FoT, paused, blacklist-against-marketplace, and non-standard return tokens at config time.

RETEST

Fixed
By capturing balBefore / balance-after deltas around the buyer's safeTransferFrom in _settleSale and reverting with Marketplace__InsufficientReceived when the actual receipt is less than gross fee-on-transfer and rebasing tokens are rejected at settle time.

M006 · ACCESS CONTROL (SCWE-016)

MEDIUM

Discount promo code is not scoped to a PackMachine cross-machine misuse + griefing

✓ FIXED

DESCRIPTION

PromoCodeRegistry.redeemDiscount(codeId,user) authorises by IPackMachineFactory(factory).isPackMachine(msg.sender). Any registered PackMachine clone can consume any Discount code. A code intended for the "Premium" machine works on the "Bargain" machine. An attacker picks the machine whose bBps × discount yields the largest profit. oncePerUser codes can be burned on the wrong machine by Sybil wallets, denying the intended audience.

AFFECTED CODE

- https://github.com/nettyworth/NettyWorthV3/blob/e2490ee60c62454780fe9e86fed9d2ecfe35a954/contracts/PromoCodeRegistry.sol
- https://github.com/nettyworth/NettyWorthV3/blob/e2490ee60c62454780fe9e86fed9d2ecfe35a954/contracts/PackMachine.sol#L848-L894

BUSINESS IMPACT

Cross-machine basis arbitrage, code-griefing burns intended audiences' redemption slots; operator per-machine promotional intent silently bypasses.

RECOMMENDED NEXT STEP

Add an address scopedMachine field to PromoCode (zero = unscoped). Enforce code.scopedMachine == address(0) || msg.sender == code.scopedMachine in redeemDiscount.

RETEST

Fixed
This vulnerability has been fixed by adding an address machine (scoped-machine) field to the PromoCode struct and enforcing bound != address(0) && msg.sender != bound in redeemDiscount (and mirrored in refundDiscount) cross-machine misuse now reverts with PromoCodeRegistry__WrongMachine.

M007 · BUSINESS LOGIC (SC02-LOGICERRORS)

MEDIUM

Auction can be cancelled after a bidder commits griefing the bidder's gas + denying the NFT

✓ FIXED

DESCRIPTION

The cancelAuction allows the seller to cancel an auction even after one or more bidders have committed on-chain bids. The bidder has already paid gas to materialise the auction state and is relying on settlement. After cancellation, state.settled = true blocks any future settleAuction. A more concerning variant: the seller signs multiple SignedAuction messages for the same NFT with different params; each can be cancelled; the seller settles whichever one is most favourable and cancels the rest, leaving honest bidders stranded across multiple auctions.

AFFECTED CODE

- https://github.com/nettyworth/NettyWorthV3/blob/e2490ee60c62454780fe9e86fed9d2ecfe35a954/contracts/NettyWorthMarketplace.sol#L362-L374
- https://github.com/nettyworth/NettyWorthV3/blob/e2490ee60c62454780fe9e86fed9d2ecfe35a954/contracts/NettyWorthMarketplace.sol#L222-L314

BUSINESS IMPACT

Bidder gas burned with no NFT; trust loss in the auction primitive. For high-value on-mainnet auctions, per-grief loss in tens of dollars.

RECOMMENDED NEXT STEP

Block seller cancellation once a bid has been committed; only MARKETPLACE_ROLE admin can force-close after a bid:

RETEST

Fixed
By adding a non-admin guard in cancelAuction if (!isAdmin && state.highestBidder != address(0)) revert Marketplace__CannotCancelWithActiveBid(...) a seller can no longer cancel once a bid has been committed; only MARKETPLACE_ROLE can force-close after a bid.

M008 · BUSINESS LOGIC (SC02-LOGICERRORS)

Default forfeits all loan interest lenders and protocol get only principal back

MEDIUM

✓ FIXED

DESCRIPTION

_initiateDefault records outstandingValue = principal not (principal + interest) and never calls _distributeInterest along any default-resolution path. Phase-1 (acquireDefaultedAsset) and Phase-2/3 (purchaseDefaultedAsset) recover only outstandingValue = principal. The interest that the borrower owed at default is permanently forfeited by both lenders and protocol. Recovery sale floor is too low: a buyer who pays exactly principal clears the loan, but the protocol's economic model assumes principal + interest would be recovered. Over many defaults, lender IRR systematically lags advertised APR.

AFFECTED CODE

- https://github.com/nettyworth/NettyWorthV3/blob/e2490ee60c62454780fe9e86fed9d2ecfe35a954/contracts/AssetLendingPool.sol#L871-L910
- https://github.com/nettyworth/NettyWorthV3/blob/e2490ee60c62454780fe9e86fed9d2ecfe35a954/contracts/AssetLendingPool.sol#L487-L533
- https://github.com/nettyworth/NettyWorthV3/blob/e2490ee60c62454780fe9e86fed9d2ecfe35a954/contracts/AssetLendingPool.sol#L543-L574

BUSINESS IMPACT

Lenders realised APR systematically below advertised; protocol fee revenue reduced; bad-credit borrowers' interest debt is silently absorbed by lenders + protocol; adverse-selection pressure.

RECOMMENDED NEXT STEP

Set outstandingValue \= principal + loan.interest in _initiateDefault; on recovery (purchaseDefaultedAsset) split outstandingValue and call _distributeInterest($, loan.interest) like a normal repay. For acquireDefaultedAsset (no USDC inflow), defer the interest distribution until the eventual pack-machine sell-through generates USDC back to the pool.

RETEST

Fixed
This issue is fixed By storing interestValue \= loan.interest on the DefaultRecord at _initiateDefault and passing it to _distributeInterest inside _resolveAndRecredit on recovery interest is no longer forfeited on default; it is preserved and split between lenders and protocol at acquisition / auction settlement.

M009 · DENIAL OF SERVICE (SCWE-087)

MEDIUM

FIXED

financeMarketplacePurchase allows self-financing of pool's defaulted collateral DoS of default resolution + permanent accounting corruption

DESCRIPTION

The financeMarketplacePurchase accepts seller as a user-supplied parameter and calls assetNFT.transferFrom(seller, address(this), tokenId) without checking the seller != address(this). After a loan defaults via _initiateDefault, the pool owns the NFT (state Held) and tokenIdToActiveLoan[tokenId] \= 0. An attacker calls financeMarketplacePurchase(tokenId, depositAmount, term, seller=address(pool)): the pool-to-pool self-transfer succeeds as a no-op (msg.sender is pool, pool is owner), _originateLoan transitions state Held Loaned and creates a new loan with the attacker as borrower on top of an already-defaulted asset. Both acquireDefaultedAsset (Phase 1) and purchaseDefaultedAsset (Phase 2/3) now revert at transferFrom because _beforeTokenTransfers enforces state \== Held. The original defaultRecord is permanently stuck as resolved=false: even after admin eventually resolves the attacker's loan default (recycling the NFT to PackMachine), the original record can never be reconciled because the NFT is no longer in the pool, so any future acquireDefaultedAsset/purchaseDefaultedAsset on the original loanId reverts at approve/transferFrom.

AFFECTED CODE

- https://github.com/nettyworth/NettyWorthV3/blob/e2490ee60c62454780fe9e86fed9d2ecfe35a954/contracts/AssetLendingPool.sol#L272-L337
- https://github.com/nettyworth/NettyWorthV3/blob/e2490ee60c62454780fe9e86fed9d2ecfe35a954/contracts/AssetLendingPool.sol#L306
- https://github.com/nettyworth/NettyWorthV3/blob/e2490ee60c62454780fe9e86fed9d2ecfe35a954/contracts/AssetLendingPool.sol#L849
- https://github.com/nettyworth/NettyWorthV3/blob/e2490ee60c62454780fe9e86fed9d2ecfe35a954/contracts/AssetLendingPool.sol#L487-L533
- https://github.com/nettyworth/NettyWorthV3/blob/e2490ee60c62454780fe9e86fed9d2ecfe35a954/contracts/AssetLendingPool.sol#L543-L574
- https://github.com/nettyworth/NettyWorthV3/blob/e2490ee60c62454780fe9e86fed9d2ecfe35a954/contracts/AssetNFT.sol#L623-L666

BUSINESS IMPACT

Permanent DoS of admin's Phase-1 acquisition path and public Phase-2/3 purchase path for any defaulted NFT under attack. Original defaultRecord stuck resolved=false indefinitely; totalDeposited permanently reduced by the original principal (never re-credited); attacker's depositAmount accumulates as untracked hidden cash in the pool that no standard withdrawal path can extract; lender liquidity (totalDeposited − totalBorrowed) artificially reduced even though the pool literally holds the cash.

RECOMMENDED NEXT STEP

Single-line guard at the top of financeMarketplacePurchase:

```
if (seller == address(this)) revert AssetLendingPool__InvalidSeller();
```

Defense-in-depth: extend _checkEligibility to reject tokens with an unresolved default record. Add mapping(uint256 tokenId \=> bool) defaultUnresolved; set it in _initiateDefault and clear it in acquireDefaultedAsset/purchaseDefaultedAsset. Then reject any new loan origination (_borrow, financeMarketplacePurchase) against tokens flagged defaultUnresolved. Note: the C-03 remediation (EIP-712 sale-consent signature on financeMarketplacePurchase) also blocks this attack as a side effect the pool is a contract and cannot ECDSA-sign for itself but an explicit seller != address(this) guard is clearer and cheaper.

RETEST

Fixed

This issue is fixed by adding if (listing.seller \== address(this)) revert AssetLendingPool__InvalidSeller(); at the top of financeMarketplacePurchase the pool now rejects any attempt to self-finance its own defaulted collateral.

M010 · UNCAUGHT EXCEPTIONS (SCWE-004)

MEDIUM

✓ FIXED

PackMachineFactory.beforeTransfer / .afterTransfer low-level .call() silently swallows validator reverts; selector 0x931820d7 does not match canonical CTS

DESCRIPTION

beforeTransfer and afterTransfer invoke the configured transfer validator via (bool ok,) \= transferValidator.call(abi.encodeWithSelector(0x931820d7, _msgSender(), token)); ok;. The trailing ok; is warning-suppression, not a check a validator revert is captured in the discarded boolean and never propagates. The inline comment ("a revert from the validator will propagate automatically") is factually wrong for .call(). Two independent defects: (1) the swallowed revert makes the Creator-Token-Standard hook a no-op; (2) validation via cast keccak "beforeAuthorizedTransfer(address,address)" yields 0x50793315, NOT the encoded 0x931820d7 the selector itself doesn't match any canonical CTS variant, so even if the swallow were fixed the call would hit a non-existent function on a compliant validator. Currently mitigated because AssetNFT._beforeTokenTransfers independently invokes ITransferValidator(validator).validateTransfer(...) via a typed high-level call that DOES propagate reverts, so token-level policy still applies on all AssetNFT transfers. But the factory-side defense-in-depth layer (intended for validators that expose only the CTS-style hook) is entirely non-functional.

AFFECTED CODE

- https://github.com/nettyworth/NettyWorthV3/blob/e2490ee60c62454780fe9e86fed9d2ecfe35a954/contracts/PackMachineFactory.sol#L204-L238
- https://github.com/nettyworth/NettyWorthV3/blob/e2490ee60c62454780fe9e86fed9d2ecfe35a954/contracts/AssetNFT.sol#L657-L668

BUSINESS IMPACT

No direct exploit against the current deployment because AssetNFT still enforces validateTransfer on every transfer. But: (a) any validator that implements only the Creator-Token-Standard interface (not validateTransfer) cannot block pack-machine-initiated transfers; (b) the misleading comment invites reviewers to trust a broken guard. Defense-in-depth is broken by design and by construction.

RECOMMENDED NEXT STEP

Fix both defects: (1) either use a typed high-level call (ITransferValidator(v).beforeAuthorizedTransfer(...)) or check the boolean and manually re-revert with the returndata (if (!ok) { assembly { revert(add(ret, 0x20), mload(ret)) } }); (2) verify the selector against the actual validator interface canonical beforeAuthorizedTransfer(address,address) is 0x50793315.

RETEST

Fixed

This vulnerability has been fixed. PackMachineFactory.beforeTransfer now uses the canonical Creator Token-Standard selector 0x50793315 (beforeAuthorizedTransfer(address,address)) and afterTransfer uses 0x0ad38899 (afterAuthorizedTransfer(address)); both branches capture the returndata on .call() failure and re-revert via assembly with the validator's original revert payload validator reverts are no longer silently swallowed and the selector now matches the canonical CTS interface.

M011 · INCONSISTENT ACCOUNTING (SCWE-010)

MEDIUM

✓ FIXED

PackMachine.resetEffectivePrizePoolSize ignores in-flight VRF reservations; post-reset opens oversell inventory, later callbacks emit CardFailed(tokenId=0)

DESCRIPTION

resetEffectivePrizePoolSize recomputes the counter by summing $.tierPools[t].length across all five tiers and assigns to $.effectivePrizePoolSize. It requires paused() but pause only gates new openPack/openPackWithPermit2 calls (whenNotPaused at those entrypoints); fulfillRandomness has NO whenNotPaused, so already-queued VRF callbacks continue to arrive during the paused window and continue to pop tokens from tierPools. Meanwhile _requestVRF has already decremented effectivePrizePoolSize by cardsPerPack at request time, so at reset the physical tierPools still contain the un-fulfilled reserved tokens while the counter has already reserved them. Reset re-adds them via the sum, inflating the counter above the true available inventory. Concrete: totalInventory=5, cardsPerPack=3, tier 0 has 5 tokens, effectivePrizePoolSize=5. User A opens □ counter 5□2. Admin pauses. Reset: sum \= 5, counter \= 5. Admin unpauses. User B opens □ _assertOpenable passes (5 ≥ 3) □ counter 5□2. VRF A fulfills first: pops 3 tokens, tier length 5□2. VRF B fulfills: pulls 2 tokens; the third card iteration finds every tier empty □ totalActiveWeight \== 0 □ fallback path emits CardFailed(user, tokenId=0, requestId) and increments the counter but does NOT refund the user.

AFFECTED CODE

- https://github.com/nettyworth/NettyWorthV3/blob/e2490ee60c62454780fe9e86fed9d2ecfe35a954/contracts/PackMachine.sol#L671-L685
- https://github.com/nettyworth/NettyWorthV3/blob/e2490ee60c62454780fe9e86fed9d2ecfe35a954/contracts/PackMachine.sol#L377-L464

BUSINESS IMPACT

Any user whose pack open lands after a reset-during-pending-VRF gets short-changed by up to cardsPerPack cards with no refund path. Requires admin misuse of the escape hatch (e.g., resetting during a Chainlink delay instead of after the queue drains) but the code silently permits it.

RECOMMENDED NEXT STEP

Track a pendingRequestCount (increment in _requestVRF, decrement in fulfillRandomness) and either forbid reset when pendingRequestCount > 0 or subtract pendingRequestCount × cardsPerPack from the recomputed value.

RETEST

Fixed
The issue has been fixed. PackMachineStorage now tracks pendingRequestCount, incremented in _requestVRF and decremented at the end of every fulfillRandomness. resetEffectivePrizePoolSize reverts with PackMachine__PendingRequests(pendingRequestCount) when in-flight VRF requests exist, so the escape hatch can no longer be misused during a Chainlink delay to inflate the counter above the actual reserved inventory.

M012 · BUSINESS LOGIC (SC02-LOGICERRORS)

MEDIUM

NettyWorthMarketplace.hashAuction returns domain-separated hash but commitBid uses bare struct hash; documented API always fails

✓ FIXED

DESCRIPTION

The public view hashAuction(SignedAuction) returns _hashTypedDataV4(_hashAuction(auction)) the fully domain-separated EIP-712 digest. Inside commitBid, auctionId is derived as bytes32 auctionId \= _hashAuction(auction); the bare struct hash without domain separation. The subsequent guard if (bid.auctionId != auctionId) revert Marketplace__InvalidSignature(); compares the bidder's supplied auctionId against the bare hash. Any bidder who follows the interface docstring which states hashAuction computes "the auctionId for a given SignedAuction (EIP-712 struct hash)" uses the view to populate bid.auctionId, then signs the bid. Every such bid reverts at the equality check. Two independent bugs: (a) the view/write hash form mismatch, (b) the docstring says "struct hash" while the view returns the domain-separated digest.

AFFECTED CODE

- https://github.com/nettyworth/NettyWorthV3/blob/e2490ee60c62454780fe9e86fed9d2ecfe35a954/contracts/NettyWorthMarketplace.sol#L222-L316
- https://github.com/nettyworth/NettyWorthV3/blob/e2490ee60c62454780fe9e86fed9d2ecfe35a954/contracts/NettyWorthMarketplace.sol#L453-L460

BUSINESS IMPACT

Every honest bidder following the documented on-chain hashing view produces bids that always revert. This is a functional break of the auction path via the documented integration route bidders must either reverse-engineer the actual hash computation or run their own EIP-712 hashing off-chain to succeed. No fund loss, no unauthorised access; correctness/DoS bug.

RECOMMENDED NEXT STEP

Either (a) change hashAuction to return the bare _hashAuction(auction) matching the docstring and the on-chain identifier, or (b) change commitBid to use _hashTypedDataV4(_hashAuction(auction)) as the auctionId (bigger surface change but aligns with EIP-712 replay-protection intent by binding the ID to the domain).

RETEST

Fixed

The issue has been fixed by option (b). commitBid now derives auctionId as _hashTypedDataV4(_hashAuction(auction)) and hashAuction returns the same value the view and write paths now use identical hashing, so bidders following the documented integration route produce bids whose auctionId matches the on-chain identifier and no longer revert at the equality check.

M013 · DENIAL OF SERVICE (SCWE-087)

MEDIUM

FIXED

Transfer-validator revert in PackMachine.fulfillRandomness permanently locks user USDC and stucks pool accounting

DESCRIPTION

PackMachine.fulfillRandomness calls iFactory.beforeTransfer(assetNFT) at :514 and iFactory.afterTransfer(assetNFT) at :611 both OUTSIDE any try/catch. PackMachineFactory.beforeTransfer (:240-258) and afterTransfer (:261-278) perform a low-level .call to the configured transfer validator and, on failure, execute assembly revert(add(revertData, 32), mload(revertData)) (:252-254, :272-274) bubbling the revert up. This bubble-up is the intended M010 fix behaviour for Creator-Token-Standard compliance. However, if the validator reverts (misconfiguration, upgraded validator with a bug, blacklist add mid-flight, oracle failure), the entire fulfillRandomness callback reverts. Solidity rolls back all state changes in the callback including totalEscrowed -= pending.escrowedAmount (:618) and pendingRequestCount-- (:659). Chainlink VRF v2.5 does NOT retry on consumer revert the request is marked fulfilled, the subscription is charged, and the callback never runs again. Post-revert state: (1) the user's escrowed USDC remains counted in $.totalEscrowed permanently; (2) pendingRequestCount stays elevated, so resetEffectivePrizePoolSize at :1466-1467 reverts forever with PackMachine__PendingRequests(N); (3) rescueERC20 (:907-917) subtracts totalEscrowed before allowing withdrawal and returns early if balance <= escrowed, so the admin cannot rescue user funds; (4) no adminForceRefundPendingOpen / adminClearRequest escape hatch exists (grep for "adminForce", "adminClear", "clearPending" in PackMachine.sol returns nothing). The interaction of the M010 validator bubble-up with the M011 count guard and the rescueERC20 escrow floor produces an unrecoverable liveness failure that did not exist before commit 7b954da.

AFFECTED CODE

- https://github.com/nettyworth/NettyWorthV3/blob/7b954dae98a99a2399dcef315203b7add7df6557/contracts/PackMachine.sol#L482-L680
- https://github.com/nettyworth/NettyWorthV3/blob/7b954dae98a99a2399dcef315203b7add7df6557/contracts/PackMachine.sol#L907-L917
- https://github.com/nettyworth/NettyWorthV3/blob/7b954dae98a99a2399dcef315203b7add7df6557/contracts/PackMachine.sol#L1454-L1470
- https://github.com/nettyworth/NettyWorthV3/blob/7b954dae98a99a2399dcef315203b7add7df6557/contracts/PackMachineFactory.sol#L240-L278

BUSINESS IMPACT

Direct permanent loss of all users' escrowed USDC affected by the failed request. Machine's prize-pool slot permanently reserved. Machine's ability to reset effective pool size permanently broken. Chainlink subscription still charged for a request that yields nothing. Because the failure is unrecoverable and no admin path exists, a single validator misconfiguration incident permanently degrades the affected PackMachine.

RECOMMENDED NEXT STEP

Add an admin escape hatch gated by paused state, DEFAULT_ADMIN_ROLE, and a minimum staleness age:

```
function adminForceRefundPendingOpen(uint256 requestId)
    external
    whenPaused
    onlyProtocolRole(Roles.DEFAULT_ADMIN_ROLE)
{
    PendingOpen memory p = _getStorage().pendingOpens[requestId];
    if (p.buyer == address(0)) revert PackMachine__UnknownRequest();
    if (block.timestamp < p.requestTimestamp + MIN_STUCK_AGE)
        revert PackMachine__RequestNotStuck();
    _getStorage().totalEscrowed -= p.escrowedAmount;
    _getStorage().pendingRequestCount--;
    _restoreTierCounts(p.tierWeightsSnapshot);
    IERC20(_getStorage().paymentToken).safeTransfer(p.buyer, p.escrowedAmount);
    delete _getStorage().pendingOpens[requestId];
    emit PendingOpenRefunded(requestId, p.buyer, p.escrowedAmount);
}
```

The whenPaused gate forces the operator to pause the machine before recovery (defense-in-depth against racing a slow-but-honest fulfillment). MIN_STUCK_AGE (suggested 24 h) prevents adminForceRefundPendingOpen from being used against a request that could still be fulfilled by Chainlink. Alternatively, wrap the beforeTransfer/afterTransfer calls in try/catch inside fulfillRandomness so a validator revert does not roll back the whole callback but that changes the M010 contract with the Creator Token Standard and needs separate consideration.

RETEST

Fixed

This issue has been fixed in commit 48934e6 (audit finding report) by implementing the recommended admin escape hatch.

M014 · BUSINESS LOGIC (SC02-LOGICERRORS)

MEDIUM

BuybackPool pause silently strips buyback rights from every card won during the pause window

✓ FIXED

DESCRIPTION

PackMachine.fulfillRandomness wraps the IBuybackPool.registerToken(...) call in try/catch at PackMachine.sol:585-594. On any failure including a whenNotPaused revert from BuybackPool the catch fires, emits BuybackRegistrationFailed(tokenId, requestId), and continues with delivery. The NFT transfer to the winner has already succeeded at :571 by the time the buyback registration is attempted. When BuybackPool is paused (an admin action by PAUSER_ROLE), every registerToken overload at :234, :247, :260 (all gated by whenNotPaused) reverts, is silently swallowed, and the resulting card is delivered to the buyer WITHOUT any TokenBuybackInfo record. The user's buyback allocation share of the pack price was still transferred from PackMachine into BuybackPool as part of the pack sale (see PackMachine.sol:626 for the buyback allocation transfer), so the user has already paid for buyback rights they did not receive. No admin retry path exists in BuybackPool.sol registerToken is only callable by an address in registeredPackMachines (i.e., an active PackMachine clone), and PackMachine only ever calls it once per card, inside the VRF callback. There is no adminRegisterToken(tokenId, tier, sourceMachine, amount) function. The buyback registration miss is permanent.

AFFECTED CODE

- https://github.com/nettyworth/NettyWorthV3/blob/7b954dae98a99a2399dcef315203b7add7df6557/contracts/PackMachine.sol#L571-L630
- https://github.com/nettyworth/NettyWorthV3/blob/7b954dae98a99a2399dcef315203b7add7df6557/contracts/BuybackPool.sol#L234-L266

BUSINESS IMPACT

Users lose the buyback allocation share they paid for up to buybackAllocationBps of the pack price is transferred into BuybackPool but the corresponding card has no on-chain claim on that value. Effect scales with pause duration × pack sales rate. Because the failure is silent from the user's perspective (only an on-chain event on the PackMachine, no user-facing revert), affected users cannot even distinguish their card from a properly-registered one on-chain; they discover the loss only when a later buyback call reverts BuybackPool__TokenNotRegistered or equivalent. Pause is a routine admin action (upgrades, incident response), so triggering conditions are commonplace.

RECOMMENDED NEXT STEP

drop whenNotPaused from the three registerToken overloads at BuybackPool.sol:234, :247, :260 registration is a bookkeeping write, not a fund transfer, so pausing it serves no safety purpose. executeBuyback can and should remain paused independently to freeze redemptions during incidents. Alternative: add an admin retry function:

```
function adminRegisterToken(
    uint256 tokenId,
    uint8 tier,
    address sourceMachine,
    uint128 amountPaidPerCard
) external onlyProtocolRole(Roles.DEFAULT_ADMIN_ROLE) {
    _registerToken(tokenId, tier, sourceMachine, amountPaidPerCard);
}
```

gated by DEFAULT_ADMIN_ROLE with off-chain rate limiting, allowing the operator to backfill missed registrations after the pause ends. Option (a) is preferred because it eliminates the failure mode entirely rather than adding a manual remediation path.

RETEST

Fixed
This issue has been fixed in commit 48934e6 by adopting the preferred remediation.

L001 · LACK OF INPUT VALIDATION (SC05-LACK OF INPUT VALIDATION)

LOW

missing zero-address check + non-pause-gated; pool unset silently disables buyback registration

✓ FIXED

DESCRIPTION

PackMachine.setBuybackPool(address pool) does not validate pool != address(0) and is not pause-gated. If the pool is unset (or set to zero), _handlePayment skips the buyback transfer and routes the full discountedPrice to financeWallet, while fulfillRandomness sets poolActive \= false and skips registerToken. Net effect: users still pay the buyback share, but the cards they win have no buyback right because they were never registered.

AFFECTED CODE

- https://github.com/nettyworth/NettyWorthV3/blob/e2490ee60c62454780fe9e86fed9d2ecfe35a954/contracts/PackMachine.sol#L605-L611
- https://github.com/nettyworth/NettyWorthV3/blob/e2490ee60c62454780fe9e86fed9d2ecfe35a954/contracts/PackMachine.sol#L884-L892
- https://github.com/nettyworth/NettyWorthV3/blob/e2490ee60c62454780fe9e86fed9d2ecfe35a954/contracts/PackMachine.sol#L391-L449

BUSINESS IMPACT

Silent UX-breaking misconfiguration: cards minted while pool is unset are never registered and can never be bought back.

RECOMMENDED NEXT STEP

Add pause and zero address validation.

```
function setBuybackPool(address pool) external onlyProtocolRole(Roles.PACK_OPERATOR_ROLE) {
    if (!paused()) revert PackMachine__NotPaused();
    if (pool == address(0)) revert PackMachine__ZeroAddress();
    ...
}
```

RETEST

Fixed
The issue is fixed.

L002 · CENTRALIZATION RISK

PackMachine.rescueERC20 can drain any ERC-20 balance admin foot-gun

LOW

✓ FIXED

DESCRIPTION

rescueERC20(token) sweeps balanceOf(this) of ANY ERC-20 to admin, including paymentToken. The in-code comment at the function explicitly tags this as a centralization tradeoff. Admin can drain payment-token balances stranded in the contract by VRF callback failure, promo redemption flows, etc. including funds rightfully owed to the buyback pool or treasury under correct accounting.

AFFECTED CODE

- https://github.com/nettyworth/NettyWorthV3/blob/e2490ee60c62454780fe9e86fed9d2ecfe35a954/contracts/PackMachine.sol#L661-L666

BUSINESS IMPACT

Centralization risk; admin can withdraw any token, including payment-token balances that should logically belong to users or the BuybackPool.

RECOMMENDED NEXT STEP

Lockout the payment token: if (token \== IPackMachineFactory($.factory).paymentToken()) revert PackMachine__CannotRescuePaymentToken();. Or maintain an allowlist of rescuable tokens.

RETEST

Fixed
This issue is fixed by floor-guarding the payment token by totalEscrowed inside rescueERC20 (with totalEscrowed tracked/updated on every VRF request and fulfillment) the sweep can no longer touch user funds pending VRF settlement. Mechanism differs from the suggested blanket block but achieves the intended user-funds protection.

L003 · MISSING EVENT EMISSION (SCWE-063)

LOW

BuybackPool _redeposit silently no-ops if source machine deregistered NFT stuck, payout already made

✓ FIXED

DESCRIPTION

The _redeposit checks isPackMachine(sourceMachine); if the source PackMachine has been deregistered by the factory admin between the user winning the card and selling it back, _redeposit returns early the NFT stays in BuybackPool with no event emitted, while the user's USDC payout has already happened. Admin can rescueNFT to recover, but only if they notice. The asymmetry creates an invisible drift: USDC out, NFT in, no signal.

AFFECTED CODE

- https://github.com/nettyworth/NettyWorthV3/blob/e2490ee60c62454780fe9e86fed9d2ecfe35a954/contracts/BuybackPool.sol#L721-L746

BUSINESS IMPACT

Stuck NFTs accumulate silently; admin needs out-of-band signal to know rescue is needed.

RECOMMENDED NEXT STEP

Emit a TokenStuck(tokenId, sourceMachine) event in the no-op branch so off-chain tooling can flag for admin rescue.

RETEST

Fixed
This issue is fixed.

L004 · SIGNATURE REPLAY ATTACKS (SCWE-055)

LOW

oncePerUser discount code burnable by anyone with leaked OpenPack signature

✓ FIXED

DESCRIPTION

openPack(address user, bytes sig, bytes32 codeId) allows msg.sender != user. The play signature only authorises users to receive cards at a given nonce; it does not bind codeId. Anyone with a valid signature for user X can call openPack(X, sig, X_ONCE_USE_CODE) paying for X's pack, gifting the cards to X, but burning X's oncePerUser redemption flag on the code. If the code was a high-value limited-redemption resource, the user is denied future redemption.

AFFECTED CODE

- https://github.com/nettyworth/NettyWorthV3/blob/e2490ee60c62454780fe9e86fed9d2ecfe35a954/contracts/PackMachine.sol#L221-L249
- https://github.com/nettyworth/NettyWorthV3/blob/e2490ee60c62454780fe9e86fed9d2ecfe35a954/contracts/PromoCodeRegistry.sol

BUSINESS IMPACT

Mild griefing victim loses oncePerUser eligibility for the consumed code; attacker pays for the pack.

RECOMMENDED NEXT STEP

Bind codeId into the EIP-712 OpenPack typehash so a signature is tied to a specific code (or bytes32(0) for no-code opens), OR restrict openPack so msg.sender \== user.

RETEST

Fixed
This issue has been fixed.

L005

LOW

NettyWorthMarketplace._executeSale royalty deducted from seller proceeds but not paid when royaltyReceiver \== address(0); USDC stranded

✓ FIXED

DESCRIPTION

_executeSale computes sellerProceeds \= gross - collectibleFee - royalty - loanDebt unconditionally, pulls gross from the buyer, and pays out the royalty inside a guard if (royalty > 0 && royaltyReceiver != address(0)). If a collection's royaltyInfo returns (receiver=0, amount>0), the amount is deducted from sellerProceeds but no safeTransfer fires the amount is silently retained in the marketplace balance forever (marketplace has no rescueERC20 only pause, config setters, and UUPS upgrade). Numeric: gross=100, fee=5, royalty=30, receiver=0, loanDebt=0 → treasury 5, seller 65, contract retains 30 permanently. Under the canonical AssetNFT this is unreachable because OZ v5.6.1 _setDefaultRoyalty/_setTokenRoyalty revert on address(0) and _deleteDefaultRoyalty leaves state at (0, 0) so the outer royalty > 0 short-circuits. Reachable only if admin allowlists a non-conforming third-party collection whose ERC-2981 implementation returns (0, non-zero) for burned/uninitialized tokens.

AFFECTED CODE

- https://github.com/nettyworth/NettyWorthV3/blob/e2490ee60c62454780fe9e86fed9d2ecfe35a954/contracts/NettyWorthMarketplace.sol#L474-L558

-

BUSINESS IMPACT

Per-sale silent loss to the seller equal to the phantom royalty amount, on any allowlisted collection whose ERC-2981 returns a non-zero amount with a zero receiver. No path to recover the retained USDC without a marketplace UUPS upgrade adding a sweep function. Unreachable against the canonical AssetNFT.

RECOMMENDED NEXT STEP

Set royalty \= 0 (or refund the residual to the seller) BEFORE computing sellerProceeds when royaltyReceiver \== address(0). Alternatively try/catch the royaltyInfo and treat zero-receiver returns as zero royalty at extraction time, not payout time.

RETEST

Fixed
The issue has been fixed.

L006 · BUSINESS LOGIC (SC02-LOGICERRORS)

LOW

AssetLendingPool.setDefaultLifecycleConfig retroactive window change contradicts docstring; admin can flip phase for in-flight defaults

✓ FIXED

DESCRIPTION

DefaultRecord stores only {loanId, tokenIds, outstandingValue, defaultedAt, resolved} no per-record window snapshot. acquireDefaultedAsset, purchaseDefaultedAsset, and getDefaultPhase all live-read $.acquisitionWindow and $.auctionWindow at every phase check. Meanwhile the setter's own docstring at AssetLendingPoolConfig.sol L347-351 promises: "Changes apply to defaults initiated after this call; existing defaults retain the windows that were active at the time of initiateDefault." Verbatim opposite of the implementation. Concrete: default R at T with acquisitionWindow \= 24h. At T+25h a public buyer's purchaseDefaultedAsset(loanId) is pending in the mempool. Owner front-runs with setDefaultLifecycleConfig(365d, 7d) now 25h < 365d, buyer's tx reverts NotInPurchasePhase. Admin's acquireDefaultedAsset(loanId, ownedMachine, tier) succeeds. Direction of harm is bidirectional (admin can also shrink windows to close Phase-2 early). No direct fund theft buyer's funds never move, so severity is opportunity-cost only.

AFFECTED CODE

- https://github.com/nettyworth/NettyWorthV3/blob/e2490ee60c62454780fe9e86fed9d2ecfe35a954/contracts/AssetLendingPoolConfig.sol#L347-L360
- https://github.com/nettyworth/NettyWorthV3/blob/e2490ee60c62454780fe9e86fed9d2ecfe35a954/contracts/AssetLendingPool.sol#L487-L560
- https://github.com/nettyworth/NettyWorthV3/blob/e2490ee60c62454780fe9e86fed9d2ecfe35a954/contracts/AssetLendingPool.sol#L670-L685

BUSINESS IMPACT

Public Phase-2 buyers can be denied at admin's discretion via retroactive window extension; admin's Phase-1 acquisition can revert via retroactive shrinkage. Integrators and users who trust the docstring are misinformed about the invariant.

RECOMMENDED NEXT STEP

Snapshot acquisitionWindow and auctionWindow into DefaultRecord at _initiateDefault and read those snapshots in every phase check.

RETEST

Fixed
The issue has been fixed.

L007 · ACCESS CONTROL (SCWE-016)

BuybackPool.initializeV2 unprotected reinitializer(2); V2 slot front-runnable

LOW

✓ FIXED

DESCRIPTION

initializeV2 is declared external reinitializer(2) with no role guard. After admin upgrades to the V2 implementation (while _initialized \== 1), any EOA can front-run the admin's post-upgrade initializeV2 call, consuming the version-2 reinitializer slot. The body writes only constants (__EIP712_init("NettyWorthBuyback", "1"), defaultBuybackModel \= AmountSpent, modelEnabled[AmountSpent] \= true, modelEnabled[FMV] \= true), which match admin intent so no direct fund loss but the reinitializer(2) slot is permanently burned. Any legitimate need to re-run V2 init (domain-separator quirk, mistaken argument in a future variant) forces escalation to a V3 impl with reinitializer(3). Attack window is bounded to the interval between the upgrade tx mining and the admin's own init tx mining; using upgradeToAndCall(newImpl, initializeV2.selector) (atomic) fully neutralises the finding, so severity depends on the deployment script.

AFFECTED CODE

- https://github.com/nettyworth/NettyWorthV3/blob/e2490ee60c62454780fe9e86fed9d2ecfe35a954/contracts/BuybackPool.sol#L247-L256

BUSINESS IMPACT

Post-upgrade griefing that consumes the version-2 reinit slot without changing final state. No direct fund loss; forces future admin re-init work to escalate to a fresh version.

RECOMMENDED NEXT STEP

Add onlyProtocolRole(Roles.DEFAULT_ADMIN_ROLE) to initializeV2 OR require deployment via upgradeToAndCall(newImpl, abi.encodeCall(BuybackPool.initializeV2, ())) (atomic upgrade + init).

RETEST

Fixed
This issue is fixed.

L008 · UNCAUGHT EXCEPTIONS (SCWE-004)

LOW

FIXED

BuybackPool._executeBuyback silently switched BuybackExecuted.payout and totalPaidOut from net to gross; off-chain integrators overstate user proceeds

DESCRIPTION

The new buyback-fee mechanism introduced in commit 91c127c splits every buyback payout into sellerAmount \= payout − fee (routed to the caller) and fee (routed to financeWallet), but leaves the pre-existing BuybackExecuted event signature and the internal totalPaidOut accumulator unchanged. _executeBuyback now writes $.totalPaidOut += payout (gross, including fee) and emits BuybackExecuted(tokenId, caller, payout, basis) with the gross payout, while the seller only ever receives payout − fee. Before this commit both surfaces were 1-to-1 with the seller's USDC delta; post-commit they overstate seller proceeds by exactly fee \= (payout × buybackFeeBps) / BPS whenever buybackFeeBps > 0. The companion BuybackFeeCharged(tokenId, feeWallet, fee) event is emitted separately, but any subgraph / analytics / frontend that keys off BuybackExecuted.payout to render a user's realised proceeds has no schema hint that it must now subtract the fee side-event. totalPaidOut is not exposed by a getter and not read anywhere on-chain, so the impact is bounded to off-chain interpretation, but the drift is monotone and silent every buyback with a non-zero fee widens the gap between the reported "paid out" number and the sum of what sellers actually received.

AFFECTED CODE

- https://github.com/nettyworth/NettyWorthV3/blob/91c127cf68074f3a0abe3167bb2e7b7b4dc95503/contracts/BuybackPool.sol#L636-L670
- https://github.com/nettyworth/NettyWorthV3/blob/91c127cf68074f3a0abe3167bb2e7b7b4dc95503/contracts/BuybackPool.sol#L651
- https://github.com/nettyworth/NettyWorthV3/blob/91c127cf68074f3a0abe3167bb2e7b7b4dc95503/contracts/BuybackPool.sol#L670

BUSINESS IMPACT

Off-chain integrations (subgraphs, indexers, user-facing dashboards, accounting exports) that treat BuybackExecuted.payout as "USDC delivered to the seller" over-report user proceeds by the fee amount, and any external consumer that later exposes totalPaidOut inherits the same drift. No on-chain funds are at risk; the discrepancy is a semantic/observability defect. Downstream harm is limited to misleading UI (users see a "paid X" number that does not match their wallet delta) and misclassified protocol-revenue vs. user-proceeds accounting.

RECOMMENDED NEXT STEP

Extend BuybackExecuted with explicit uint256 sellerAmount and uint256 fee fields (or emit a new event and deprecate the old one) so integrators have unambiguous per-buyback net/gross values. Split $.totalPaidOut into a totalSellerPaid accumulator and a totalFeesCollected accumulator, and expose both via view getters so off-chain systems can key off the correct signal without reconciling two separate events.

RETEST

Fixed
This issue has been fixed.

DISCLOSURE

The Reports provided by CredShields are not an endorsement or condemnation of any specific project or issue and do not guarantee the security of any specific project. The contents of this report are not intended to be used to make decisions about buying or selling tokens, products, services, or any other assets and should not be interpreted as such.

Emerging technologies such as Smart Contracts and Solidity carry a high level of technical risk and uncertainty. CredShields does not provide any warranty or representation about the quality of code, the business model or the proprietors of any such business model, or the legal compliance of any business. The report is not intended to be used as investment advice and should not be relied upon as such.

CredShields Audit team is not responsible for any decisions or actions taken by any third party based on the report.

REVIEWER(S) & CONTACT

Shashank

Co-founder, CredShields

LEAD REVIEWERS & REPORT SIGNATORIES



Shashank

Co-founder & CEO · CredShields

Security analyst and penetration tester with **12 years of experience** in pentesting and **5 years** in managing bug bounty programs. Listed in **Google, Facebook, Apple,** and 30+ websites' hall of fame for reporting valid security issues.

CVE-2017-7063 CVE-2017-7062 CVE-2017-2458



Aditya Dixit

Audit & Research Lead · CredShields

Senior security researcher and penetration tester with **10+ years** in cybersecurity. Led pentest teams at **Cobalt and HackerOne**; previously at Binary.com. Executed **500+ pentests** across banking, fintech, payments, healthcare, telecom, and media, covering web, mobile (iOS/Android), network, cloud, code review, and smart contracts.

OSCP CREST CPSA AWS Security Specialty AWS Cloud Practitioner Qualys VMDR